POGLAVLJE 76

C++

Sumeja Halilović, Delila Halilović, Samir Ribić

76.1 Osnovne osobine jezika

C++ je programski jezik opšte namjene, koga je kreirao Bjarne Stroustrup kao proširenje programskog jezika C poznat kao ”C sa klasama”.

76.1.1 Vrsta i namjena

Programska paradigma određuje stil programiranja, pogled koji programer ima nad programom i njegovih izvršavanjem. Kao primjer, u objektnom programiranju programer razmišlja o programu kao o grupi objekata koji međusobno komuniciraju, dok u proceduralnom programiranju na program se gleda kao redoslijed naredbi. Programski jezik C++ podržava tzv. objektno orijentisano programiranje što je programerima omogućilo da lakše definišu kompleksnije programe. Jezik C++ bio je jedan od prvih sa podrškom za klase (način na koji se definišu objekti). U C++ je moguće pored dominantnog objektno orijentisanog programiranja, programirati i proceduralno, nestruktuirano, pa i funkcionalno (jezik više paradigmi).

C ++ je dizajniran sa orijentacijom prema sistemskom programiranju i ugrađenom softveru s ograničenim resursima i velikim sistemima, sa performansama, efikasnošću i fleksibilnošću upotrebe kao najvažnijim elementima dizajna. C++ se također pokazao korisnim u mnogim drugim kontekstima, a ključne su mu snage softverska infrastruktura i uključujući stone aplikacije, video igre, servere (npr. E-trgovina, web pretraga ili SQL serveri) i performansno kritične primjene.

76.1.2 Istorijat

Godine 1979. Bjarne Stroustrup, danski naučnik iz Bell Labsa, započeo je razvoj C++ jezika kojeg je prvo nazvao ”C” s klasama koji je bio preteća za C++. Poticaj za stvaranje novog programskog jezika proisteklo je iz njegovog rada na dokorskoj disertaciji u kojem se susreo s dilemom gdje je programski jezik Simula bio dobar za složene programske projekte, dok je programski jezik BCPL bio brz ali je bio na jako niskoj razini da bude praktičan za primjenu. Kada je Stroustrup počeo raditi za AT&Tove Bell laboratorije, i kada je počeo analizirati jezgru operativnog sistema UNIX za distribuirano računarstvo. Za taj problem primijenio je svoje iskustvo iz svog doktorskog rada, tako što je počeo dodavati produžetke koje postoje u jeziku Simula za programski jezik C, kojeg je on koristio jer je bio dostupan, brz i lako prenosiv na druge platforme. Ovaj

produženi C programski jezik, prvo se zvao "C s klasama", ali će formalno dobiti naziv C++ krajem 1983, uzimajući da je "++" znak za povećavanje u programskom jeziku C. U oktobru 1985. godine prva komercijalna distribucija jezika predstavljena je javnosti u knjizi The C++ Programming Language čiji je autor spomenuti Bjarne Stroustrup.

1989. godine objavljen je C++ 2.0, a nakon njega ažurirano drugo izdanje programskog jezika C++ 1991. godine. Nove osobine 2.0 uključivale su višestruko nasljeđivanje, apstraktne klase, statičke funkcije člana, članske **const** funkcije i zaštićene članove. 1990. godine objavljen je Anotirani priručnik za C ++. Ovaj rad postao je osnova za budući standard. Kasniji dodaci uključivali su predloške, iznimke, prostore imena, nove uloge i logički tip.

1998. godine objavljen je C ++ 98, koji standardizira jezik, a manja nadogradnja (C++ 03) objavljena je 2003. godine.

Nakon C ++ 98, C ++ se relativno sporo razvijao, sve dok 2011. nije objavljen C++ 11 standard, dodajući brojne nove funkcije, dodatno proširujući standardnu biblioteku i pružajući više mogućnosti programerima C++. Nakon manjeg ažuriranja C++ 14 objavljenog u decembru 2014., u C++ 17 uvedeni su razni novi dodaci. Standard C++ 20 tehnički je finaliziran u februaru 2020, a nacrt je odobren 4. septembra 2020.

76.1.3 Način implementacije

Kompajliranje je prevođenje programa u višem programskom jeziku (izvorni jezik) u mašinski jezik. Osobina ovog pristupa je da prevođenje zahtijeva izvjesno vrijeme, ali izvršavanje je dosta brzo, jer su programi prevedeni u mašinski jezik. Kod interpretera se dijelovi programa napisani na nekom programskom jeziku, nakon sintaksne provjere, odmah izvršavaju. Za razliku od kompajlera, ne mora cijeli izvorni program da bude prvo preveden u drugi programski jezik. Jezik C++ koristi kompajlere, mada sadrži i dosta interpretera kao što su CINT, ch interpreter.

76.1.4 Format pisanja programa

Programi u C++ se pišu u tekstualnom ASCII formatu sa slobodnim prelascima u nove redove osim što se ne može prekinuti leksem. Unicode znakovi se mogu nalaziti u string literalima. Uvlačenje je korisno radi preglednosti, ali nije obavezno. Terminator naredbi je ";", a složene naredbe se pišu u vitičastim zagradama { }. Kao komentar koriste se znakovi /* i */ između kojih se ignoriše sav tekst ili // gdje se ignoriše tekst do kraja linije. Dodavanjem predprocesorske direktive #include kompletan sadržaj napisane datoteke sa izvornim kodom se ubaci u tekst programa, kao da je otkucan, što se najčešće koristi za činjenje eksternih funkcija dostupnim.

76.2 Sintaksa i semantika

C++ je proširenje jezika C, pa se većina C koda lako može kompajlirati u C ++. Neki od glavnih dodataka C-u uključuju objektno orijentirano programiranje, preopterećenje operatora, višestruko nasljeđivanje i rukovanje iznimkama. Posljedica je da se jezik karakteriše prilično kompleksnom sintaksom, pogotovo kada se koriste template.

76.2.1 Prikaz gramatike

Slijedi gramatika jezika C++, verzija ISO/IEC 14882:1998 u EBNF formatu. Gramatika je LR tipa i nije kompatibilna sa automatskim generatorima parsera.

```
abstract_declarator  ::=          ptr_operator [ abstract_declarator ]
 |  direct_abstract_declarator

access_specifier  ::=   'private' |  'protected' |  'public'

```

```
additive_expression  ::=   multiplicative_expression
 |  additive_expression '+' multiplicative_expression
 |  additive_expression '-' multiplicative_expression

and_expression  ::=   equality_expression
 |  and_expression '&' equality_expression

asm_definition  ::='asm' '(' string_literal ')' ';'

assignment_expression  ::=   conditional_expression
 |  logical_or_expression assignment_operator assignment_expression
 |  throw_expression

assignment_operator  ::=   '=' |  '*=' |  '/=' |  '%=' |  '+=' |  '-='
                         |  '>>=' |  '<<=' |  '&=' |  '^=' |  '|='

base_clause  ::=':' base_specifier_list

base_specifier  ::=   [ '::' ] [ nested_name_specifier ] class_name
 |  'virtual' [ access_specifier ] [ '::' ] [ nested_name_specifier ] class_name
 |  access_specifier [ 'virtual' ] [ '::' ] [ nested_name_specifier ] class_name

base_specifier_list  ::=   base_specifier
 |  base_specifier_list ',' base_specifier

block_declaration  ::=   simple_declaration
 |  asm_definition
 |  namespace_alias_definition
 |  using_declaration
 |  using_directive

boolean_literal  ::=   'false' |  'true'

c_char  ::=   ANY_CHARACTER_EXCEPT_NEWLINE_SQUOTE_BACKSLASH
 |  escape_sequence
 |  universal_character_name

c_char_sequence  ::=   c_char
 |  c_char_sequence c_char

cast_expression  ::=   unary_expression
 |  '(' type_id ')' cast_expression

character_literal  ::=   "'" c_char_sequence "'"
 |  "L'" c_char_sequence "'"

class_head  ::=   class_key [ identifier ] [ base_clause ]
 |  class_key nested_name_specifier identifier [ base_clause ]
 |  class_key [ nested_name_specifier ] template_id [ base_clause ]

class_key  ::=   'class' |  'struct' |  'union'

class_name  ::=   identifier
 |  template_id

class_or_namespace_name  ::=   class_name |  namespace_name

class_specifier  ::=class_head '{' [ member_specification ] '}'

compound_statement  ::='{' [ statement_seq ] '}'

condition  ::=   expression
 |  type_specifier_seq declarator '=' assignment_expression
```

```
69
70 conditional_expression  ::=   logical_or_expression
71  |  logical_or_expression '?' expression ':' assignment_expression
72
73 constant_expression  ::=conditional_expression
74
75 constant_initializer  ::='=' constant_expression
76
77 control_line  ::=   '#' 'include' pp_tokens new_line
78  |  '#' 'define' identifier replacement_list new_line
79  |  '#' 'define' identifier lparen [ identifier_list ] ')' replacement_list new_line
80  |  '#' 'undef' identifier new_line
81  |  '#' 'line' pp_tokens new_line
82  |  '#' 'error' [ pp_tokens ] new_line
83  |  '#' 'pragma' [ pp_tokens ] new_line
84  |  '#' new_line
85
86 conversion_declarator  ::=ptr_operator [ conversion_declarator ]
87
88 conversion_function_id  ::='operator' conversion_type_id
89
90 conversion_type_id  ::=type_specifier_seq [ conversion_declarator ]
91
92 ctor_initializer  ::=':' mem_initializer_list
93
94 cv_qualifier  ::=   'const' |  'volatile'
95
96 cv_qualifier_seq  ::=cv_qualifier [ cv_qualifier_seq ]
97
98 decimal_literal  ::=   nonzero_digit |  decimal_literal digit
99
100 decl_specifier  ::=   storage_class_specifier
101  |  type_specifier
102  |  function_specifier
103  |  'friend'
104  |  'typedef'
105
106 decl_specifier_seq  ::=[ decl_specifier_seq ] decl_specifier
107
108 declaration  ::=   block_declaration
109  |  function_definition
110  |  template_declaration
111  |  explicit_instantiation
112  |  explicit_specialization
113  |  linkage_specification
114  |  namespace_definition
115
116 declaration_seq  ::=   declaration
117  |  declaration_seq declaration
118
119 declaration_statement  ::=block_declaration
120
121 declarator  ::=   direct_declarator
122  |  ptr_operator declarator
123
124 declarator_id  ::=   id_expression
125  |  [ '::' ] [ nested_name_specifier ] type_name
126
127 delete_expression  ::=
128    [ '::' ] 'delete' cast_expression
129  |  [ '::' ] 'delete' '[' ']' cast_expression
130
131 digit  ::=DIGIT
```

```
132
133 digit_sequence  ::=   digit
134  |  digit_sequence digit
135
136 direct_abstract_declarator  ::=
137    [ direct_abstract_declarator ] '(' paramerter_declaration_clause ')' [
          cv_qualifier_seq ] [ exception_specification ]
138  |  [ direct_abstract_declarator ] '[' [ constant_expression ] ']'
139  |  '(' abstract_declarator ')'
140
141 direct_declarator  ::=   declarator_id
142  |  direct_declarator '(' paramerter_declaration_clause ')' [ cv_qualifier_seq ] [
         exception_specification ]
143  |  direct_declarator '[' [ constant_expression ] ']'
144  |  '(' declarator ')'
145
146 direct_new_declarator  ::=   '[' expression ']'
147  |  direct_new_declarator '[' constant_expression ']'
148
149 elaborated_type_specifier  ::=   class_key [ '::' ] [ nested_name_specifier ]
        identifier
150  |  'enum' [ '::' ] [ nested_name_specifier ] identifier
151  |  'typename' [ '::' ] nested_name_specifier identifier
152  |  'typename' [ '::' ] nested_name_specifier [ 'template' ] template_id
153
154 elif_group  ::='#' 'elif' constant_expression new_line [ group ]
155
156 elif_groups  ::=   elif_group
157  |  elif_groups elif_group
158
159 else_group  ::='#' 'else' new_line [ group ]
160
161 endif_line  ::='#' 'endif' new_line
162
163 enum_name  ::=identifier
164
165 enum_specifier  ::='enum' [ identifier ] '{' [ enumerator_list ] '}'
166
167 enumerator  ::=identifier
168
169 enumerator_definition  ::=   enumerator
170  |  enumerator '=' constant_expression
171
172 enumerator_list  ::=   enumerator_definition
173  |  enumerator_list ',' enumerator_definition
174
175 equality_expression  ::=   relational_expression
176  |  equality_expression '==' relational_expression
177  |  equality_expression '!=' relational_expression
178
179 escape_sequence  ::=   simple_escape_sequence
180  |  octal_escape_sequence
181  |  hexadecimal_escape_sequence
182
183 exception_declatation  ::=   type_specifier_seq declarator
184  |  type_specifier_seq abstract_declarator
185  |  type_specifier_seq
186  |  '...'
187
188 exception_specification  ::='throw' '(' [ type_id_list ] ')'
189
190 exclusive_or_expression  ::=   and_expression
191  |  exclusive_or_expression '^' and_expression
```

```
192
193 explicit_instantiation  ::='template' declaration
194
195 explicit_specialization  ::='template' '<' '>' declaration
196
197 exponent_part  ::=   'e' [ sign ] digit_sequence
198  |  'E' [ sign ] digit_sequence
199
200 expression  ::=   assignment_expression
201  |  expression ',' assignment_expression
202
203 expression_list  ::=   assignment_expression
204  |  expression_list ',' assignment_expression
205
206 expression_statement  ::=[ expression ] ';'
207
208 extension_namespace_definition  ::='namespace' original_namespace_name '{'
        namespace_body '}'
209
210 floating_literal  ::=   fractional_constant [ exponent_part ] [ floating_suffix ]
211  |  digit_sequence exponent_part [ floating_suffix ]
212
213 floating_suffix  ::=   'f' |  'l' |  'F' |  'L'
214
215 for_init_statement  ::=   expression_statement
216  |  simple_declaration
217
218 fractional_constant  ::=   [ digit_sequence ] '.' digit_sequence
219  |  digit_sequence '.'
220
221 function_body  ::=compound_statement
222
223 function_definition  ::=   [ decl_specifier_seq ] declarator [ ctor_initializer ]
        function_body
224  |  [ decl_specifier_seq ] declarator function_try_block
225
226 function_specifier  ::=   'inline' |  'virtual' |  'explicit'
227
228 function_try_block  ::='try' [ ctor_initializer ] function_body handler_seq
229
230 group  ::=   group_part |  group group_part
231
232 group_part  ::=   [ pp_tokens ] new_line
233  |  if_section
234  |  control_line
235
236 h_char  ::=ANY_CHARACTER_EXCEPT_NEWLINE_GT
237
238 h_char_sequence  ::=   h_char
239  |  h_char_sequence h_char
240
241 handler  ::='catch' '(' exception_declatation ')' compound_statement
242
243 handler_seq  ::=handler [ handler_seq ]
244
245 header_name  ::=   '<' h_char_sequence '>'
246  |  '"' q_char_sequence '"'
247
248 hex_quad  ::=hexadecimal_digit hexadecimal_digit hexadecimal_digit hexadecimal_digit
249
250 hexadecimal_digit  ::=   '0' |  '1' |  '2' |  '3' |  '4' |  '5' |  '6' |  '7' |  '8'
        |  '9' |  'a'
251  | 'b' |  'c' |  'd' |  'e' |  'f' |  'A' |  'B' |  'C' |  'D' |  'E' |  'F'
```

```
252
253 hexadecimal_escape_sequence  ::=   '\x' hexadecimal_digit
254  |  hexadecimal_escape_sequence hexadecimal_digit
255
256 hexadecimal_literal  ::=   '0x' hexadecimal_digit
257  |  '0X' hexadecimal_digit
258  |  hexadecimal_literal hexadecimal_digit
259
260 id_expression  ::=   unqualified_id |  qualified_id
261
262 identifier  ::=   nondigit
263  |  identifier nondigit
264  |  identifier digit
265
266 identifier_list  ::=   identifier
267  |  identifier_list ',' identifier
268
269 if_group  ::=   '#' 'if' constant_expression new_line [ group ]
270  |  '#' 'ifdef' identifier new_line [ group ]
271  |  '#' 'ifndef' identifier new_line [ group ]
272
273 if_section  ::=if_group [ elif_groups ] [ else_group ] endif_line
274
275 inclusive_or_expression  ::=   exclusive_or_expression
276  |  inclusive_or_expression '|' exclusive_or_expression
277
278 init_declarator  ::=declarator [ initializer ]
279
280 init_declarator_list  ::=   init_declarator
281  |  init_declarator_list ',' init_declarator
282
283 initializer  ::=   '=' initializer_clause |  '(' expression_list ')'
284
285 initializer_clause  ::=   assignment_expression
286  |  '{' initializer_list [ ',' ] '}'
287  |  '{' '}'
288
289 initializer_list  ::=   initializer_clause
290  |  initializer_list ',' initializer_clause
291
292 integer_literal  ::=   decimal_literal [ integer_suffix ]
293  |  octal_literal [ integer_suffix ]
294  |  hexadecimal_literal [ integer_suffix ]
295
296 integer_suffix  ::=   unsigned_suffix [ long_suffix ]
297  |  long_suffix [ unsigned_suffix ]
298
299 iteration_statement  ::=   'while' '(' condition ')' statement
300  |  'do' statement 'while' '(' expression ')' ';'
301  |  'for' '(' for_init_statement [ condition ] ';' [ expression ] ')' statement
302
303 jump_statement  ::=   'break' ';'
304  |  'continue' ';'
305  |  'return' [ expression ] ';'
306  |  'goto' identifier ';'
307
308 labeled_statement  ::=   identifier ':' statement
309  |  'case' constant_expression ':' statement
310  |  'default' ':' statement
311
312 linkage_specification  ::=   'extern' string_literal '{' [ declaration_seq ] '}'
313  |  'extern' string_literal declaration
314
```

```
literal  ::=   integer_literal
 |  character_literal
 |  floating_literal
 |  string_literal
 |  boolean_literal

logical_and_expression  ::=   inclusive_or_expression
 |  logical_and_expression '&&' inclusive_or_expression

logical_or_expression  ::=   logical_and_expression
 |  logical_or_expression '||' logical_and_expression

long_suffix  ::=   'l' |  'L'

lparen  ::=LEFT_PARENTHESIS_WITHOUT_LEADING_WS

mem_initializer  ::=mem_initializer_id '(' [ expression_list ] ')'

mem_initializer_id  ::=   [ '::' ] [ nested_name_specifier ] class_name
 |  identifier

mem_initializer_list  ::=   mem_initializer
 |  mem_initializer ',' mem_initializer_list

member_declaration  ::=   [ decl_specifier_seq ] [ member_declarator_list ] ';'
 |  function_definition [ ';' ]
 |  [ '::' ] nested_name_specifier [ 'template' ] unqualified_id ';'
 |  using_declaration
 |  template_declaration

member_declarator  ::=   declarator [ pure_specifier ]
 |  declarator [ constant_initializer ]
 |  [ identifier ] ':' constant_expression

member_declarator_list  ::=   member_declarator
 |  member_declarator_list ',' member_declarator

member_specification  ::=   member_declaration [ member_specification ]
 |  access_specifier ':' [ member_specification ]

multiplicative_expression  ::=   pm_expression
 |  multiplicative_expression '*' pm_expression
 |  multiplicative_expression '/' pm_expression
 |  multiplicative_expression '%' pm_expression

named_namespace_definition  ::=   original_namespace_definition
 |  extension_namespace_definition

namespace_alias  ::=identifier

namespace_alias_definition  ::='namespace' identifier '='
    qualified_namespace_specifier ';'

namespace_body  ::=[ declaration_seq ]

namespace_definition  ::=   named_namespace_definition
 |  unnamed_namespace_definition

namespace_name  ::=   original_namespace_name
 |  namespace_alias

nested_name_specifier  ::=
   class_or_namespace_name '::' [ nested_name_specifier ]
```

```
 |  class_or_namespace_name '::' 'template' nested_name_specifier

new_declarator  ::=   ptr_operator [ new_declarator ]
 |  direct_new_declarator

new_expression  ::=
    [ '::' ] 'new' [ new_placement ] new_type_id [ new_initializer ]
 |  [ '::' ] 'new' [ new_placement ] '(' type_id ')' [ new_initializer ]

new_initializer  ::='(' [ expression_list ] ')'

new_line  ::=NEWLINE

new_placement  ::='(' expression_list ')'

new_type_id  ::=type_specifier_seq [ new_declarator ]

nondigit  ::=   universal_character_name
 |  ALPHA_CHARACTER
 |  '_'

nonzero_digit  ::=   '1' |  '2' |  '3' |  '4' |  '5' |  '6' |  '7' |  '8' |  '9'

octal_digit  ::=   '0' |  '1' |  '2' |  '3' |  '4' |  '5' |  '6' |  '7'

octal_escape_sequence  ::=   '\'␣octal_digit
␣|␣␣'\'␣octal_digit␣octal_digit
␣|␣␣'\'␣octal_digit␣octal_digit␣octal_digit

octal_literal␣␣::=␣␣␣'0'
␣|␣␣octal_literal␣octal_digit

operator␣␣::=␣␣␣'new'␣|␣␣'delete'␣|␣␣'new[]'␣|␣␣'delete[]'
␣|␣␣'+'␣|␣␣'-'␣|␣␣'*'␣|␣␣'/'␣|␣␣'%'␣|␣␣'^'␣|␣␣'&'␣|␣␣'|'
␣|␣␣'~'␣|␣␣'!'␣|␣␣'='␣|␣␣'<'␣|␣␣'>'␣|␣␣'+='␣|␣␣'-='␣|␣␣'*='
␣|␣␣'/='␣|␣␣'%='␣|␣␣'^='␣|␣␣'&='␣|␣␣'|='␣|␣␣'<<'␣|␣␣'>>'␣|␣␣'>>='
␣|␣␣'<<='␣|␣␣'=='␣|␣␣'!='␣|␣␣'<='␣|␣␣'>='␣|␣␣'&&'␣|␣␣'||'␣|␣␣'++'
␣|␣␣'--'␣|␣␣','␣|␣␣'->*'␣|␣␣'->'␣|␣␣'()'␣|␣␣'[]'

operator_function_id␣␣::='operator'␣operator

original_namespace_definition␣␣::='namespace'␣identifier␣'{'␣namespace_body␣'}'

original_namespace_name␣␣::=identifier

paramerter_declaration_clause␣␣::=␣␣␣[␣parameter_declaration_list␣]␣[␣'...'␣]
␣|␣␣parameter_declaration_list␣','␣'...'

parameter_declaration␣␣::=␣␣␣decl_specifier_seq␣declarator
␣|␣␣decl_specifier_seq␣declarator␣'='␣assignment_expression
␣|␣␣decl_specifier_seq␣[␣abstract_declarator␣]
␣|␣␣decl_specifier_seq␣[␣abstract_declarator␣]␣'='␣assignment_expression

parameter_declaration_list␣␣::=␣␣␣parameter_declaration
␣|␣␣parameter_declaration_list␣','␣parameter_declaration

pm_expression␣␣::=␣␣␣cast_expression
␣|␣␣pm_expression␣'.*'␣cast_expression
␣|␣␣pm_expression␣'->*'␣cast_expression

postfix_expression␣␣::=␣␣␣primary_expression
␣|␣␣postfix_expression␣'['␣expression␣']'
␣|␣␣postfix_expression␣'('␣[␣expression_list␣]␣')'
```

```
␣|␣␣simple_type_specifier␣'('␣[␣expression_list␣]␣')'
␣|␣␣'typename'␣[␣'::'␣]␣nested_name_specifier␣identifier␣'('␣[␣expression_list␣]␣')'
␣|␣␣'typename'␣[␣'::'␣]␣nested_name_specifier␣[␣'template'␣]␣template_id␣'('␣[␣
    expression_list␣]␣')'
␣|␣␣postfix_expression␣'.'␣[␣'template'␣]␣id_expression
␣|␣␣postfix_expression␣'->'␣[␣'template'␣]␣id_expression
␣|␣␣postfix_expression␣'.'␣pseudo_destructor_name
␣|␣␣postfix_expression␣'->'␣pseudo_destructor_name
␣|␣␣postfix_expression␣'++'
␣|␣␣postfix_expression␣'--'
␣|␣␣'dynamic_cast'␣'<'␣type_id␣'>'␣'('␣expression␣')'
␣|␣␣'static_cast'␣'<'␣type_id␣'>'␣'('␣expression␣')'
␣|␣␣'reinterpret_cast'␣'<'␣type_id␣'>'␣'('␣expression␣')'
␣|␣␣'const_cast'␣'<'␣type_id␣'>'␣'('␣expression␣')'
␣|␣␣'typeid'␣'('␣expression␣')'
␣|␣␣'typeid'␣'('␣type_id␣')'

pp_number␣␣::=␣␣␣digit
␣|␣␣'.'␣digit
␣|␣␣pp_number␣digit
␣|␣␣pp_number␣nondigit
␣|␣␣pp_number␣'e'␣sign
␣|␣␣pp_number␣'E'␣sign
␣|␣␣pp_number␣'.'

pp_tokens␣␣::=␣␣␣preprocessing_token
␣|␣␣pp_tokens␣preprocessing_token

preprocessing_file␣␣::=[␣group␣]

preprocessing_op_or_punc␣␣::=␣␣␣'{'␣|␣␣'}'␣|␣␣'['␣|␣␣']'␣|␣␣'#'
␣|'##'␣|␣␣'('␣|␣␣')'␣|␣␣'<:'
␣|␣␣':>'␣|␣␣'<%'␣|␣␣'%>'␣|␣␣'%:'␣|␣␣'%:%:'␣|␣␣';'␣|␣␣':'␣|␣␣'...'
␣|␣␣'new'␣|␣␣'delete'␣|␣␣'?'␣|␣␣'::'␣|␣␣'.'␣|␣␣'.*'␣|␣␣'+'␣|␣␣'-'
␣|␣␣'*'␣|␣␣'/'␣|␣␣'%'␣|␣␣'^'␣|␣␣'&'␣|␣␣'|'␣|␣␣'~'␣|␣␣'!'␣|␣␣'='
␣|␣␣'<'␣|␣␣'>'␣|␣␣'+='␣|␣␣'-='␣|␣␣'*='␣|␣␣'/='␣|␣␣'%='␣|␣␣'^='␣|␣␣'&='
␣|␣␣'|='␣|␣␣'<<'␣|␣␣'>>'␣|␣␣'>>='␣|␣␣'<<='␣|␣␣'=='␣|␣␣'!='␣|␣␣'<='␣|␣␣'>='
␣|␣␣'&&'␣|␣␣'||'␣|␣␣'++'␣|␣␣'--'␣|␣␣','␣|␣␣'->*'␣|␣␣'->'␣|␣␣'and'
␣|␣␣'and_eq'␣|␣␣'bitand'␣|␣␣'bitor'␣|␣␣'compl'␣|␣␣'not'␣|␣␣'not_eq'␣|␣␣'or'
␣|␣␣'or_eq'␣|␣␣'xor'␣|␣␣'xor_eq'

preprocessing_token␣␣::=␣␣␣header_name
␣|␣␣identifier
␣|␣␣pp_number
␣|␣␣character_literal
␣|␣␣string_literal
␣|␣␣preprocessing_op_or_punc
␣|␣␣ANY_NOT_YET_COVERED_NON_WS_CHARACTER

primary_expression␣␣::=␣␣␣literal
␣|␣␣'this'
␣|␣␣'('␣expression␣')'
␣|␣␣id_expression

pseudo_destructor_name␣␣::=␣␣␣[␣'::'␣]␣[␣nested_name_specifier␣]␣type_name␣'::'␣'~'␣
    type_name
␣|␣␣[␣'::'␣]␣nested_name_specifier␣'template'␣template_id␣'::'␣'~'␣type_name
␣|␣␣[␣'::'␣]␣[␣nested_name_specifier␣]␣'~'␣type_name

ptr_operator␣␣::=␣␣␣'*'␣[␣cv_qualifier_seq␣]
␣|␣␣'&'
␣|␣␣[␣'::'␣]␣nested_name_specifier␣'*'␣[␣cv_qualifier_seq␣]

```

```
pure_specifier␣␣::=␣␣'='␣'0'

q_char␣␣::=ANY_CHARACTER_EXCEPT_NEWLINE_DQUOTE

q_char_sequence␣␣::=␣␣␣q_char
␣|␣␣␣q_char_sequence␣q_char

qualified_id␣␣::=␣␣␣␣[␣'::'␣]␣nested_name_specifier␣[␣'template'␣]␣unqualified_id
␣|␣␣␣'::'␣identifier
␣|␣␣␣'::'␣operator_function_id
␣|␣␣␣'::'␣template_id

qualified_namespace_specifier␣␣::=␣[␣'::'␣]␣[␣nested_name_specifier␣]␣namespace_name

relational_expression␣␣::=␣␣␣␣shift_expression
␣|␣␣␣relational_expression␣'<'␣shift_expression
␣|␣␣␣relational_expression␣'>'␣shift_expression
␣|␣␣␣relational_expression␣'<='␣shift_expression
␣|␣␣␣relational_expression␣'>='␣shift_expression

replacement_list␣␣::=␣[␣pp_tokens␣]

s_char␣␣::=␣␣␣␣ANY_CHARACTER_EXCEPT_NEWLINE_DQUOTE_BACKSLASH
␣|␣␣␣escape_sequence
␣|␣␣␣universal_character_name

s_char_sequence␣␣::=␣␣␣␣s_char
␣|␣␣␣s_char_sequence␣s_char

selection_statement␣␣::=␣␣␣␣'if'␣'('␣condition␣')'␣statement
␣|␣␣␣'if'␣'('␣condition␣')'␣statement␣'else'␣statement
␣|␣␣␣'switch'␣'('␣condition␣')'␣statement

shift_expression␣␣::=␣␣␣␣additive_expression
␣|␣␣␣shift_expression␣'<<'␣additive_expression
␣|␣␣␣shift_expression␣'>>'␣additive_expression

sign␣␣::=␣␣␣␣'+'␣|␣␣␣'-'

simple_declaration␣␣::=[␣decl_specifier_seq␣]␣[␣init_declarator_list␣]␣';'

simple_escape_sequence␣␣::=␣␣␣␣"\'"␣|␣␣␣'\"'␣|␣␣␣'\?'␣|␣␣␣'\\'␣|
'\a'␣|␣␣␣'\b'␣|␣␣␣'\f'␣|␣␣␣'\n'␣|␣␣␣'\r'␣|␣␣␣'\t'␣|␣␣␣'\v'

simple_type_specifier␣␣::=␣␣␣␣[␣'::'␣]␣[␣nested_name_specifier␣]␣type_name
␣|␣␣␣[␣'::'␣]␣nested_name_specifier␣'template'␣template_id
␣|␣␣␣'char'␣|␣␣␣'wchar_t'␣|␣␣␣'bool'␣|␣␣␣'short'␣|␣␣␣'int'␣|␣␣␣'long'
␣|␣␣␣'signed'␣|␣␣␣'unsigned'␣|␣␣␣'float'␣|␣␣␣'double'␣|␣␣␣'void'

statement␣␣::=␣␣␣␣labeled_statement
␣|␣␣␣expression_statement
␣|␣␣␣compound_statement
␣|␣␣␣selection_statement
␣|␣␣␣iteration_statement
␣|␣␣␣jump_statement
␣|␣␣␣declaration_statement
␣|␣␣␣try_block

statement_seq␣␣::=␣␣␣␣statement
␣|␣␣␣statement_seq␣statement

storage_class_specifier␣␣::=␣␣␣␣'auto'␣|␣␣␣'register'␣|␣␣␣'static'
␣|␣␣␣'extern'␣|␣␣␣'mutable'
```

```
564
565 string_literal␣␣::=␣␣␣'"'␣[␣s_char_sequence␣]␣'"'
566 ␣|␣␣'L"'␣[␣s_char_sequence␣]␣'"'
567
568 syntax␣␣::=␣␣␣cpp_syntax_translation_unit
569 ␣|␣␣preprocessor_syntax_preprocessing_file
570
571 template_argument␣␣::=␣␣␣assignment_expression
572 ␣|␣␣type_id
573 ␣|␣␣id_expression
574
575 template_argument_list␣␣::=␣␣␣template_argument
576 ␣|␣␣template_argument_list␣','␣template_argument
577
578 template_declaration␣␣::=␣[␣'export'␣]␣'template'␣'<'␣template_parameter_list␣'>'␣
    declaration
579
580 template_id␣␣::=␣template_name␣'<'␣[␣template_argument_list␣]␣'>'
581
582 template_name␣␣::=identifier
583
584 template_parameter␣␣::=␣␣␣type_parameter
585 ␣|␣␣parameter_declaration
586
587 template_parameter_list␣␣::=␣␣␣template_parameter
588 ␣|␣␣template_parameter_list␣','␣template_parameter
589
590 throw_expression␣␣::='throw'␣[␣assignment_expression␣]
591
592 token␣(not␣used)␣␣::=␣␣␣identifier
593 ␣|␣␣KEYWORD
594 ␣|␣␣literal
595 ␣|␣␣operator
596 ␣|␣␣PUNCTUATOR
597
598 translation_unit␣␣::=[␣declaration_seq␣]
599
600 try_block␣␣::='try'␣compound_statement␣handler_seq
601
602 type_id␣␣::=type_specifier_seq␣[␣abstract_declarator␣]
603
604 type_id_list␣␣::=␣␣␣type_id
605 ␣|␣␣type_id_list␣','␣type_id
606
607 type_name␣␣::=␣␣␣class_name␣|␣␣enum_name␣|␣␣typedef_name
608
609 type_parameter␣␣::=␣␣␣'class'␣[␣identifier␣]
610 ␣|␣␣'class'␣[␣identifier␣]␣'='␣type_id
611 ␣|␣␣'typename'␣[␣identifier␣]
612 ␣|␣␣'typename'␣[␣identifier␣]␣'='␣type_id
613 ␣|␣␣'template'␣'<'␣template_parameter_list␣'>'␣'class'␣[␣identifier␣]
614 ␣|␣␣'template'␣'<'␣template_parameter_list␣'>'␣'class'␣[␣identifier␣]␣'='␣
    id_expression
615
616 type_specifier␣␣::=␣␣␣simple_type_specifier
617 ␣|␣␣class_specifier
618 ␣|␣␣enum_specifier
619 ␣|␣␣elaborated_type_specifier
620 ␣|␣␣cv_qualifier
621
622 type_specifier_seq␣␣::=type_specifier␣[␣type_specifier_seq␣]
623
624 typedef_name␣␣::=identifier
```

```
625
626 unary_expression␣␣::=␣␣␣postfix_expression
627 ␣|␣␣'++'␣cast_expression
628 ␣|␣␣'--'␣cast_expression
629 ␣|␣␣unary_operator␣cast_expression
630 ␣|␣␣'sizeof'␣unary_expression
631 ␣|␣␣'sizeof'␣'('␣type_id␣')'
632 ␣|␣␣new_expression
633 ␣|␣␣delete_expression
634
635 unary_operator␣␣::=␣␣␣'*'␣|␣␣'&'␣|␣␣'+'␣|␣␣'-'␣|␣␣'!'␣|␣␣'~'
636
637 universal_character_name␣␣::=␣␣␣'\u'␣hex_quad
638 ␣|␣␣'\U'␣hex_quad␣hex_quad
639
640 unnamed_namespace_definition␣␣::='namespace'␣'{'␣namespace_body␣'}'
641
642 unqualified_id␣␣::=␣␣␣identifier
643 ␣|␣␣operator_function_id
644 ␣|␣␣conversion_function_id
645 ␣|␣␣'~'␣class_name
646 ␣|␣␣template_id
647
648 unsigned_suffix␣␣::=␣␣␣'u'␣|␣␣'U'
649
650 using_declaration␣␣::=␣␣'using'␣'::'␣unqualified_id␣';'
651 ␣|␣'using'␣[␣'typename'␣]␣[␣'::'␣]␣nested_name_specifier␣unqualified_id␣';'
652
653 using_directive␣␣::='using'␣'namespace'␣[␣'::'␣]␣[␣nested_name_specifier␣]␣
    namespace_name␣';'
```

76.3 Imena, vezanja i opsezi

76.3.1 Imena

Imena u C++ ne zavise od dužine. U C++ nema ograničenja, ali implementacija može uticati. U imenima su dopuštena su sva slova engleske abecede, brojevi i znak _ (podcrtavanje). Ime mora početi sa slovom ili sa znakom _ (podcrtavanje). Radi brže leksičke analize, u C++ razlikuju se velika i mala slova, tj. C++ je case sensitive. Npr. auto i Auto su dvije različite varijable.

76.3.1.1 Rezervisane i ključne riječi

Sljedeće riječi u C++ su rezervisane i ne mogu se koristiti kao imena varijabli i drugih objekata: **alignas, alignof, and, and_eq, asm, atomic_cancel, atomic_commit, atomic_noexcept, auto, bitand, bitor, bool, break, case, catch, char, char8_t, char16_t, char32_t, class, compl, concept, const, consteval, constexpr, constinit, const_cast, continue, co_await, co_return, co_yield, decltype, default, delete, do, double, dynamic_cast, else, enum, explicit, export, extern, false, float, for, friend, goto, if, inline, int, long, mutable, namespace, new, noexcept, not, not_eq, nullptr, operator, or, or_eq, private, protected, public, reflexpr, register, reinterpret_cast, requires, return, short, signed, sizeof, static, static_assert, static_cast, struct, switch, synchronized, template, this, thread_local, throw, true, try, typedef, typeid, typename, union, unsigned, using, virtual, void, volatile, wchar_t, while, xor, xor_eq.**

Sve ključne riječi su ujedno i rezervisane.

76.3.2 Vezanja i opsezi

Opseg programa je statički i ne dopuštaju se gniježdeni potprogrami. Varijable mogu biti deklarisane unutar blokova, naredbi programske strukture, na početku funkcije ili metode, u okviru klase ili globalno. Globalna varijabla deklarira se izvan bilo kojeg bloka koda i bit će joj dostupna s bilo kojeg mjesta nakon što je deklarirana, čak i u drugim izvornim datotekama u istom projektu. S druge strane, lokalna varijabla deklarira se unutar funkcije i bit će joj dostupna tek nakon što bude deklarirana. Životni vijek lokalne varijable je također ograničen. Globalna varijabla ostaće dodijeljena za vrijeme trajanja programa, dok će lokalna varijabla biti uništena kad se njena funkcija završi s izvršavanjem.

Povezivanje varijable s tipom je statičko. Nijedna varijabla se ne može koristiti bez specificiranja njenog tipa. Zahvaljujući ovom načinu povezivanja kompajler nas štiti od bilo kakvog pokušaja smještanja vrijednosti čiji tip nije kompatibilan sa tipom varijable. Kao npr smještanje stringa u int varijablu. Deklarisanje tipa prije korištenja također omogućava lako nalaženje grešaka kucanja. Npr varijabla se koristi u dosta izraza ali se pogrešno otkucala u jednom od njih. Bez deklaracije, kompajler bi to bez ikakvog upozorenja smatrao novom varijablom, a sa deklaracijom bi se pojavila greška "Undeclared Variable". Još jedan razlog ovakvog povezivanja je da kompajler zna koliko mjesta u memoriji treba biti alocirano za ovu varijablu.

Najjednostavniji primjer deklaracije (dijelovi unutar [] su opcionalni) :

```
[specifier(s)] type variable_name [ = initial_value];
```

Sintaksa za kreiranje varijable tipa int izgleda ovako:

```
int sum;
```

Ne postoje implicitno definisani statički povezani tipovi. Eksplicitna deklaracija je naredba programa koja se koristi za tipove varijabli , u jeziku C++ koristi se obavezna deklaracija varijabli navodeći ime tipa iza koga slijedi lista varijabli.

U C++ nema zaključivanja tipova ili dinamičkog dodjeljivanja tipova. Postoje konverzije tipova koje mogu biti automatske ili eksplicitne, a objektno orijentisano programiranje omogućava da da se objekat jednog tipa tretira kao objekat drugog tipa, a to omogućava polimorfizam, nasljeđivanje, ono na čemu se objektno orijentisano programiranje i zasniva.

Po vremenu života C++ razlikuje sljedeće tipove varijabli:

1. Statičke varijable su vezane za memorijske ćelije prije početka izvršenja i ostaju vezane za nju tokom izvršenja. U ovu kategoriju spadaju globalne kao i lokalne varijable u C++ koje su označene ključnom riječju static.

2. Stack-dinamičke varijable se vezuju s memorijom se kada se obrađuju njihove naredbe deklarisanja. To se dešava pri izvršenju programa, na primjer prilikom početka izvršavanja potprograma u kome je varijabla deklarisana. Ostali atributi za skalarne varijable se povezuju statički. Primjer ovakvih varijabli su lokalne varijable u C ++.

3. Eksplicitno heap-dinamičke varijable su neimenovane (apstraktne) memorijske ćelije koje se alociraju i dealociraju eksplicitnim naredbama koje se izvršavaju u toku rada programa, a piše ih programer. Heap je skup memorijskih lokacija čija je organizacija dosta neuređena zbog nepredvidljivosti upotrebe. U C++, operator alokacije, koji se zove **new**, koristi ime tipa kao svoj operand. Kada se izvrši, kreira se eksplicitna heap-dinamička varijabla čiji je tip naveden kao operand i vrati se adresa.

Sljedeći dio koda u C++ alocira heap varijable:

```
int *intnode; // Kreiraj pointer
intnode = new int; // Kreiraj heap-dynamic varijablu
/* . . .  */
delete intnode; // Dealociraj varijablu na koju intnode pokazuje
```

Varijable bi se trebale deklarisati kao lokalne i što kasnije je to moguće (pred samu upotrebu) i inicijalizirane odmah. Tri su mjesta gdje se varijable mogu deklarisati:

- Unutar funkcije ili bloka, lokalne varijable
- Unutar definisanja parametara funkcije, formalni parametri
- Izvan svih funkcija, globalne varijable.

Opseg važenja varijable unutar funkcije ili bloka je od mjesta deklarisanja pa do kraja te funkcije ili bloka. Van funkcije nije važeća kao ni unutar funkcije a prije deklaracije. Opseg važenja formalnih parametara je također unutar te funkcije kojoj su parametri. Opseg važenja globalnih varijabli je unutar čitavog programa, u svim funkcijama, blokovima itd.

Pored varijabli, postoje i imenovane konstante. Imenovana konstanta je varijabla koja dobije vrijednost samo jednom. Time program dobije na čitljivosti. C++ za razliku npr od Pascala-a dopušta izraze bilo koje vrste. Npr:

```
1  const int result = 2 * width + 1;
```

76.4 Tipovi

C++ spada u jezike sa sva sistema tipova. Jedan su osnovni tipovi naslijeđeni iz jezika C, a drugi su apstraktni tipovi, klase. U ovom poglavlju će biti riječi o osnovnim tipovima podataka.

76.4.1 Osnovni tipovi podataka

C++ razlikuje sljedeće osnovne tipove podataka: cjelobrojni, znakovni, broj u pokretnom zarezu i logički.

76.4.1.1 Cjelobrojni tipovi

Cjelobrojni tipovi podataka u C++ su refleksija mašinskih operacija nad cijelim brojevima. Zbog različitog načina implementacije operacija množenja, dijeljenja, poređenja i pomjeranja, C++ ima odvojene verzije predznačenih i nepredznačenih cijelih brojeva.

U tabeli 76.1 navedeni su cjelobrojni tipovi koje C++ poznaje. Ako nije navedena specifikacija **signed** ili **unsigned** podrazumijeva se **signed**.

Tablica 76.1: Cjelobrojni tipovi

Naziv	Opseg	Opis
signed char	Predznačeni	kao char bar 8 bita
signed short int	Predznačeni	ne manji od char bar 16 bita
signed int	Predznačeni	ne manji od short bar 16 bita
signed long int	Predznačeni	ne manji od int bar 32 bita
signed long long int	Predznačeni	ne manji od long bar 64 bita
unsigned char	Nepredznačeni	bar 8 bita
unsigned short int	Nepredznačeni	ne manji od char bar 16 bita
unsigned int	Nepredznačeni	ne manji od short bar 16 bita
unsigned long int	Nepredznačeni	ne manji od int bar 32 bita
unsigned long long int	Nepredznačeni	ne manji od long bar 64 bita

Nad cijelim brojevima mogu se koristiti aritmetičke operacije navedene u tablici 76.2.

Tablica 76.2: Aritmetički cjelobrojni operatori

Naziv	Primjer	Opis
+	a+b	sabiranje
-	a-b	oduzimanje
*	a*b	množenje
/	a/b	dijeljenje
%	a%b	ostatak pri cjelobrojnom dijeljenju
&	a&b	Binarno AND
|	a |b	Binarno OR
^	a ^b	Binarno XOR
<<	a<<b	Pomjeranje bitova lijevo
>>	a>>b	Pomjeranje bitova desno
+	+b	unarni plus
-	-b	unarni minus
~	~b	Bit negacija

Svi binarni aritmetički operatori imaju pripadne operatore pridodjeljivanja. Njihov efekt opisan je u tabeli 76.3.

Tablica 76.3: Cjelobrojni operatori pridodjeljivanja

Naziv	Sintaksa	Efekat
+=	a+=b	a=a+b
-=	a-=b	a=a-b
*=	a*=b	a=a*b
/=	a/=b	a=a/b
%=	a%=b	a= a%b
&=	a&=b	a=a&b
^=	a ^=b	a ^=b
|=	a |= b	a=a |b
<<=	a<<=b	a=a<<b
>>=	a>>=b	a=a>>b
++	a++	a=a+1 return a
++	++a	a=a+1 return a+1
–	a–	a=a-1 return a
–	–a	a=a-1 return a-1

Sljedeći primjer pokazuje kako se koriste ovi operatori.

```
1  // primjer upotrebe operatora pridodjeljivanja
2  #include <iostream>
3  using namespace std;
4  int main ()
5    {
6      int a, b=3;
7      a = b;
8      a+=2;              // ekvivalentno sa a=a+2
9      cout << a;   //ispisuje na ekranu 5
10   }
```

Operatori inkrementiranja i dekrementiranja (++, –), predstavljaju unarne operacije pridodjeljivanja gdje se parametar sabira ili oduzima sa 1. Operatori mogu biti prefiksni ili postfiksni. Primjer razlike se vidi u sljedećem dijelu koda.

```
x = 3; y = ++x;
// x je 4, y je 4
x = 3; y = x++;
// x je 4, y je 3
```

Između dva cijela broja mogu se obavljati operacije poređenja navedene u tabeli 76.4. Rezultat operacije je tipa **bool**.

Tablica 76.4: Cjelobrojni operatori poređenja

operator	opis
==	jednakost
!=	nejednakost
<	manje
>	veće
<=	manje ili jednako
>=	veće ili jednako

Ovdje vidimo nekoliko rezultata poređenja.

```
int a=2, b=3, c=6;
(a == 5);     // rezultat je false
(a*b >= c);   // true
(b+4 > a*c);  // false
((b=2) == a); // true
```

76.4.1.2 Brojevi u pokretnom zarezu

Brojevi u pokretnom zarezu predstavljaju realne vrijednosti, kao 3.14 ili 0.01, sa različitom preciznošću. Kao decimalni separator koristi se tačka. Brojevni u pokretnom zarezu su predznačeni. Ako je potrebno predstaviti male ili velike brojeve u eksponencijalnom obliku, koristi se E notacija. Na primjer, broj $-3,8 \cdot 10^{-3}$ se piše u obliku -3.8E-3. U tabeli 76.5 su predstavljene moguće preciznosti.

Tablica 76.5: Tipovi brojeva u pokretnom zarezu

Naziv	Opis
float	broj u pokretnom zarezu jednostruke preciznosti. Obično IEEE-754 32 bit FP broj
double	broj u pokretnom zarezu dvostruke preciznosti. Obično IEEE-754 64 bit FP broj
long double	broj u pokretnom zarezu proširene preciznosti

Nad brojevima u pokretnom zarezu mogu se koristiti operatori navedeni u tablici 76.6.

Tablica 76.6: Aritmetički operatori u pokretnom zarezu

Naziv	Primjer	Opis
+	a+b	sabiranje
-	a-b	oduzimanje
*	a*b	množenje
/	a/b	dijeljenje
+	+b	unarni plus
-	-b	unarni minus

Tablica 76.7: Operatori pridodjeljivanja u pokretnom zarezu

Naziv	Sintaksa	Efekat
+=	a+=b	a=a+b
-=	a-=b	a=a-be
*=	a*=b	a=a*b
/=	a/=b	a=a/b
++	a++	a=a+1 return a
++	++a	a=a+1 return a+1
–	a–	a=a-1 return a
–	–a	a=a-1 return a-1

Mogu se koristiti isti operatori poređenja koji su navedeni u 76.4.

Transcedentne funkcije nisu dio osnovne definicije jezika, ali su implementirane u standardnoj biblioteci. Uključenjem zaglavlja <cmath> postaju dostupne funkcije navedene u tabeli.

Tablica 76.8: Matematičke funkcije

Funkcija	Značenje	Funkcija	Značenje
cos(x)	$cos(x)$	sin(x)	$sin(x)$
tan(x)	$tg(x)$	acos(x)	$arccos(x)$
asin(x)	$arcsin(x)$	atan(x)	$arctg(x)$
atan2(y ,x)	$arctg(\frac{y}{x})$	cosh(x)	$cosh(x)$
sinh(x)	$sinh(x)$	tanh(x)	$tgh(x)$
acosh(x)	$arccosh(x)$	asinh(x)	$arcsinh(x)$
atanh(x)	$arctgh(x)$	exp(x)	e^x
ldexp(x ,exp)	$x \cdot 2^e xp$	log (x)	$ln(x)$
log10(x)	$log_{10}(x)$	modf(x)	$m = [x]; f = frac(x)$
exp2(x)	2^x	expm1(x)	$e^x - 1$
log1p(x)	$ln(1 + x)$	log2(x)	$log_2(x)$
pow(x ,y)	x^y	sqrt(x)	$\sqrt{x}$
cbrt(x)	$\sqrt[3]{x}$	hypot(x,y)	$\sqrt{x^2 + y^2}$
erf(x)	$erf(x)$	tgamma(x)	$\Gamma(x)$
lgamma(x)	$ln(\|\Gamma(x)\|)$	ceil(x)	$\lceil x \rceil$
floor(x)	$\lfloor x \rfloor$	fmod(x)	$xmody$
trunc(x)	$[x]$	round(x)	$[x + 0.5]$
copysign(x ,y)	$x \cdot sgn(y)$	fdim(x ,y)	$x > y \Rightarrow x - y$
fmax (x ,y)	$max(x, y)$	fmin (x ,y)	$min(x, y)$
fabs(x)	$\|x\|$	fma(x,y,z)	$x \cdot y + z$

76.4.1.3 Kompleksni brojevi

U C++ Kompleksni brojevi nisu osnovni tip. Realizovani su kao klasa u okviru standardne biblioteke. Pri upotrebi ove klase uključi se biblioteka <complex>. Sljedeći primjer pokazuje kako se ova klasa koristi.

```
#include <iostream>     // std::cout
#include <complex>      // std::complex
int main ()
  {
    std::complex<double> mycomplex;
    mycomplex = 10.0;   // 10
    mycomplex += 2.0;   // 12
    mycomplex = std::complex<double>(10.0,1.0);  // 10+i
    mycomplex = mycomplex + 10.0 ;               // 20+i
    if (mycomplex == std::complex<double>(20.0,1.0))
    std::cout << "mycomplex␣is␣" << mycomplex; // mycomplex je (20,1)
    return 0;
  }
```

76.4.1.4 Logički tip

Logički tip podataka je tip podataka koji može imati jednu od dvije vrijednosti, **true** i **false** kako bi se predstavila istinitost izraza. Logički tip se primarno povezuje sa uvjetom koji određuje dalji tok izvršavanja programa.

U C++ ovaj tip se zove **bool**. U suštini on je preimenovani cjelobrojni tip. Podrazumijevana vrijednost za **true** je 1, a za **false** je 0. Varijable tipa **bool** i vrijednosti **true** i **false** mogu se javljati u matematičkim izrazima. Na primjer, sljedeći izraz je ispravan i imaće vrijednost 7, pošto je **false** jednak 0, a **true** jednak 1.

```
int x = false + true + 6;
```

Ako **bool** varijabli dodijelimo cjelobrojnu ili FP vrijednost, vrijednost 0 će se smatrati false, a ostale vrijednosti true.

```
bool x = 0;  // false
bool y = 100;  // true
bool z = 15.75;  // true
```

Nad **bool** tipom mogu se raditi logičke operacije. Operator **&&** (logička konjunkcija) i **||** (logička disjunkcija) vraćaju logičke vrijednosti kako je predstavljeno u tabeli 76.9.

Tablica 76.9: Operatori konjunkcije i disjunkcije

a	b	a && b	a ||b
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

Operator logičke negacije **!** je unarna operacija, koja invertuje bool vrijednost.

Tablica 76.10: Operator logičke negacije

a	b
true	false
false	true

Logičke operacije se kombinuju sa relacionim operacijama.

```
//primjer
((5 == 5) && (3 > 6)) ;  // vrijednost  false (true && false)
((5 == 5) || !(3 > 6 ) ; // vrijednost true (true || ! false)
```

76.4.2 Znakovni tipovi

Podaci tipa **char** pamte individualne znakove. Individualni znakovi se pišu unutar jednostrukih navodnika. Znakovi se u računaru kodiraju i sve manipulacije sa znakovima obavljaju se putem njihovih numeričkih kodova. Zbog toga je tip **char** ustvari cjelobrojni tip podataka vrlo malog raspona, najčešće je to jedan bajt. Na tip **char** možemo primijeniti kvalifikatore signed i unsigned kao i na ostale cjelobrojne tipove.

Ako se za **char** koristi jedan bajt, onda tip **unsigned char** poprima pozitivne cjelobrojne vrijednosti između 0 i 255, dok tip **signed char** poprima pozitivne i negativne cjelobrojne vrijednosti izmedu -128 i 127. Pravilo je da se kvalifikatori **signed** i **unsigned** ne koriste s tipom **char** ako on služi za manipulaciju sa znakovima, već samo ako ga koristimo kao cjelobrojni tip malog raspona.

Postoje različiti načini kodiranja znakova. Najčešći je ASCII kod, ali u C++ postoje i varijante za 16-bitne i 32-bitne znakove, po standardu Unicode. Četiri odgovarajuća tipa znakova data su u tabeli.

Tablica 76.11: Znakovni tipovi

Naziv tipa	Opis
char	8 bita
char16_t	16 bita
char32_t	32 bita
wchar_t	16 ili 32 bita

76.4.3 Ostali skalarni tipovi

Enumeracijski (pobrojani) tip je onaj kod koga se u sve moguće vrijednosti navode u definiciji kao imenovane konstante. U C++ nije dopušteno varijabli tipa **enum** dodijeliti cijeli broj, ali obrnuto je dopušteno.

```
1  enum colors_t {black, blue, green, cyan, red, purple, yellow, white};
2  colors_t mycolor;
3
4  mycolor = blue;
5  if (mycolor == green) mycolor = red;
```

U C++ ne postoji podintervalni tip. Također ni skupovni tip ne postoji. Funkcionalnost skupova se postiže logičkim bit operacijama nad konstantama tako odabranim da jedan bit predstavlja pripadnost skupu.

76.4.4 Znakovni nizovi

Znakovni niz u C++ nije elementarni tip, za razliku od npr. BASIC-a. Podrška za nizove znakova urađena je na dva načina, kroz nizove tipa **char** i korištenjem klase String u biblioteci STL.

76.4.4.1 Nizovi tipa char

Iz jezika C naslijeđeni su nizovi tipa **char**. Na primjer

```
1  char poruka [20];
```

Gornja deklaracija definisala je je niz koji može pohraniti do 20 elemenata tipa **char**. Ali ovaj kapacitet ne mora se u potpunosti iskoristiti: niz može primiti i kraće sekvence. Na primjer, u nekom trenutku programa, i sekvenca ”Zdravo” i sekvenca ”Javi mi se sutra” mogu biti pohranjene u niz poruka, jer bi obje mogle stati u sekvencu kapaciteta 20 znakova.

Prema dogovoru, kraj nizova predstavljenih u sekvencama znakova signalizira se posebnim znakom: **null** znakom , čija se doslovna vrijednost može zapisati kao '\0'.

Budući da su nizovi znakova obični nizovi, oni slijede ista pravila. Na primjer, da inicijaliziramo niz znakova nekim unaprijed određenim nizom znakova, to možemo učiniti baš kao i bilo koji drugi niz:

```
char myword[] = { 'Z', 'd', 'r', 'a', 'v', 'o', '\0' };
```

Gore navedeno deklarira niz od 7 elemenata tipa char inicijaliziranih znakovima koji čine riječ "Zdravo" plus nulti znak '\0' na kraju.

Ali nizovi elemenata znakova imaju još jedan način za inicijalizaciju: izravno korištenje stringova literala. String literali se navode zatvaranjem teksta između dvostrukih navodnika ("). Stoga se niz znakova može inicijalizirati null-terminiranim nizom znakova bilo kojim od ova dva izraza:

```
char niz1[] = { 'H', 'e', 'l', 'l', 'o', '\0' };
char niz2[] = "Hello";
```

U oba slučaja niz se deklarira s veličinom od 6 elemenata tipa char, od čega 5 znakova koji sastavljaju riječ "Hello", plus završni null znak ('\0'), koji specificira kraj niza i koji, u drugom slučaju, kada se koriste dvostruki navodnici (") dodaje se automatski.

```
char str[] = "abc";
```

Nad nizovima ovih znakova se ne obavljaju operacije pomoću operatora, nego se koriste pozivi funkcija iz biblioteka. Većina funkcija koje se pozivaju kao parametre imaju pokazivače na početak niza znakova. Za upotrebu ovih funkcija treba uključiti zaglavlje <cstring>. Bitnije funkcije koje pruža ovo zaglavlje date su u tabeli 76.12.

Tablica 76.12: Važnije funkcije za manipulaciju nizovima znakova

Funkcija	Namjena
strcpy	Prebacivanje sadržaja jednog niza u drugi
strncpy	Prebacivanje sadržaja jednog niza u drugi sa ograničenjem veličine
strcat	Nadovezivanje niza na drugi
strncat	Nadovezivanje niza na drugi sa ograničenjem broja znakova
strcmp	Poređenje dva niza u ASCII poretku
strcoll	Poređenje dva niza u nacionalnom poretku
strncmp	Poređenje dva niza u ASCII poretku sa ograničenjem veličine
strchr	Traženje prve pojave znaka u nizu
strstr	Traženje prve pojave podniza u nizu
strtok	Podijela niza u tokene
strlen	Račun dužine niza

76.4.4.2 Klasa string

C++ u standardnoj biblioteci ima klasu **string** koja je bolje rješenje od **char** nizova. Uključuje i odgovarajuće operatore. Klasa **string** dio je C ++ standardne biblioteke. String predstavlja niz znakova, sa dodatnim operatorima i riješenim alokacijom i dealokacijom memorije.

Da bi se koristila klasu **string**, treba uključiti zaglavlje <string>

Nizovi znakova tipa string mogu se kreirati i inicijalizirati na više načina koristeći različite konstruktore.

Tablica 76.13: Konstruktori stringova

Oblik	Namjena
string ()	kreira prazan string ("")
string (otherstring)	stvara niz identičan otherstring
string (otherstring ,position ,count)	kreira niz koji sadrži count znakova iz otherstring
string (count ,character)	kreira niz koji sadrži znak ponovljen count puta

Klasa string također ima destruktor koji se brine za oslobađanje memorije koja pohranjuje znakove kada je objekt uništen.

Dužina stringa je dinamička, može se mijenjati tokom izvršavanja programa.

Tablica 76.14: Metode stringova

Članska Funkcija	Namjena
const char * data()	vraća niz znakova u stilu C,a predstavljaju sadržaj niza
unsigned int length ()	vraća dužinu niza
unsigned int size ()	vraća dužinu niza
bool empty ()	vraća se true ako je niz prazan,a false u suprotnom
void swap (otherstring)	zamijeni sadržaj ovog niza sa sadržajem otherstring
string & append (otherstring)	dodaje otherstring ovom nizu
string & insert (position,otherstring)	ubacuje se otherstring u niz na mjestu position
string & erase (position,count)	uklanja count znakova iz niza počevši od znaka position
unsigned int find (otherstring,position)	pronalazi otherstring unutar ovog niza i vraća mu položaj
string substr (position,count)	vraća podniz koji počinje na position dužine count iz niza

Za razliku od nizova **char** nad tipom **string** se mogu koristiti operatori.

Tablica 76.15: Članske funkcije stringova

Operator	Namjena
=	Dodjela
+=	Pridodavanje
[]	Indeksiranje
+	Spajanje
==	Poređenje na jednakost
!=	Poređenje na nejednakost
>	Leksikografsko poređenje na veću poziciju u ASCII skupu
<	Leksikografsko poređenje na manju poziciju u ASCII skupu
>=	Leksikografsko poređenje na veću poziciju u ASCII skupu ili jednakost
<=	Leksikografsko poređenje na manju poziciju u ASCII skupu ili jednakost

Sljedeći kratki primjer pokazuje kreiranje, spajanje stringova i poziv metode za njegovu dužinu.

```
1  // string::size
2  using namespace std;
3  #include <iostream>
4  #include <string>
5
6  int main ()
7    {
8      string str ("Test␣string");
9      str = str+str;
10     cout << "Duzina␣str␣" << str.size() << "␣bajta.\n";
11     return 0;
12   }
```

76.4.5 Nizovi

Niz (array) je spoj homogenih elemenata podataka u kome se individualni element određuje pozicijom relativno u odnosu na prvi element. Indeksiranje (indexing, subscripting) je mapiranje indeksa na elemente

C++ koristi uglaste zagrade za pristup elementima niza. Npr. a= a + b[i], gdje je b naziv niza, a i indeks. C++ ne provjerava da li je indeks dozvoljenom opsegu prije pristupanju elementu niza. Višedimenzionalni niz se posmatra kao niz nizova, pa se svaka koordinata piše unutar posebnih zagrada, npr.

```
MyArray[3][7];
```

Što se tiče opsega indeksa, najniža vrijednost indeksa može biti 0. Ukoliko niz ima n elemenata, vrijednost posljednjeg indeksa će biti n-1. Globalno definisani nizovi u C++ i nizovi s **static** modifikatorom definisani unutar funkcije C++ su statički nizovi. Nizovi u C++ unutar funkcije bez **static** modifikatora su fiksni stek dinamički. C++ ima i fiksne heap-dinamičke nizove (koristeći funkcije **malloc**, **free** odnosno operatore **new delete**). U deklaracijama poput **int a[4][8]** realizuju se nizovi kao pravougaoni (svi redovi imaju isti broj elemenata a sve kolone imaju isti broj elemenata). Međutim, ako je riječ o nizu pointera čijem se pojedinom elementu pristupa sintaksno na isti način kao pravougaonom (poput **int * a[4]**), tada nizovi mogu biti neravnomjerni.

Ovo je inicijalizacija niza cjelobrojnih vrijednosti u C++:

```
int list [] = {4, 5, 7, 83}
```

Nizovi znakova u C++ se inicijaliziraju jednom vrijednošću:

```
char name [] = ""freddie;
```

Nizovi stringova u C++ se mogu također inicijalizirati.

```
char *names [] = {""Bob,␣""Jake, ""Joe};
```

U osnovnoj definiciji, C++ ne podržava heterogene nizove (heterogeni niz je onaj čiji elementi ne moraju biti istog tipa). U slučaju potrebe za takvom strukturom, može se koristiti niz pointera na void, niz unija ili niz objekata bazne klase.

Odsječci nizova nisu sastavni dio jezika. Standardna biblioteka poznaje drugi tip podataka, sličan nizovima, **valarray** kod kojih su moguće operacije sa odsječcima nizova.

76.4.6 Asocijativni nizovi

Asocijativni nizovi nisu dio osnovne specifikacije jezika. Međutim, pored podrške za obične nizove, standardna biblioteka C++-a uvodi i biblioteku za rad sa mapama, koje predstavljaju vrstu asocijativnih nizova. Mape čuvaju elemente u kombinaciji <ključ, vrijednost>. Sljedeći primjer pokazuje način njihovog korištenja. Primjer ispisuje **Found b**.

```
#include <iostream>
#include <map>

int main()
  {
    std::map<int,char> example = {{1,'a'},{2,'b'}};
    auto search = example.find(2); //pronalazi element sa ključem dva
    if(search != example.end()) {
        std::cout << "Found␣" << search->first << "␣" << search->second << '\
            n';
      }
    else {
        std::cout << "Not␣found\n";
      }
  }
```

76.4.7 Liste

Inicijalizaciona lista je skupina podataka unutar vitičastih zagrada koji su međusobno razdvojeni zarezima (npr. “2.5, 3.17, 4, -2.25” ili “x, y, z”). Pri tome, u inicijalizacionim listama zarez zaista služi kao separator, dakle za razdvajanje stavki. Do pojave standarda C++11, u jezicima C i C++ inicijalizacione liste su se mogle koristiti samo za inicijalizaciju nizova i struktura prilikom njihove deklaracije, i nizašta drugo. U dijalektu C++11 mogućnosti inicijalizacionih listi su izuzetno proširene i one se sada mogu koristiti i na mnogim drugim mjestima.

76.4.8 N-torke

N-torka je tip podataka sličan slogu ali se elementi ne imenuju. U C++ nisu osnovni tip, su realizovani kroz standardnu biblioteku

```
#include <tuple>
#include <iostream>
#include <string>

int main()
  {
    auto student0 = std::make_tuple(3.8, 'A', "Lisa␣Simpson");
    std::cout << "ID:␣0,␣"
    << "GPA:␣" << std::get<0>(student0) << ",␣"
    << "grade:␣" << std::get<1>(student0) << ",␣"
    << "name:␣" << std::get<2>(student0) << '\n';
    double gpa1;
    char grade1;
    std::string name1;
    std::tie(gpa1, grade1, name1) = std::make_tuple(2.9, 'C', "Milhouse␣Van␣
        Houten");
    std::cout << "ID:␣1,␣"
    << "GPA:␣" << gpa1 << ",␣"
    << "grade:␣" << grade1 << ",␣"
    << "name:␣" << name1 << '\n';
  }
```

76.4.9 Slogovi

Slog je spoj elemenata podataka različitih tipova u kome se pojedini elementi identifikuju imenom. U C, C ++ i C#, slogovi se zovu strukture i podržani su tipom **struct**. Na primjer

```
struct Books {
    char title[50];
    char author[50];
    char subject[100];
    int book_id;
  } book;
```

U C++, strukture su varijacija klase čiji su svi članovi javni. Za pristup nekom elementu strukture koristi se . (tačka), npr. knjiga.title. Ako se ima pointer na strukturu, tada se može elementu pristupiti i operatorom ->. Struktura može u sebi sadržati drugu strukturu.

76.4.10 Klase

Klase su strukture proširene operacijama i osnove su apstraktnih tipova podataka. O njima će biti riječi u posebnom poglavlju.

76.4.11 Unije

Unija (varijantni slog) je tip čijim varijablama je dopušteno da smjeste različite vrijednosti tipova u različita vremena izvršenja. Fortran, C, i C++ pružaju konstrukcije unija u kojim nema jezičke podrške za provjeru tipova. Deklarišu se ključnom rječju **union**. Ovakva unija je slobodna unija . Slobodne unije su nesigurne jer ne podržavaju provjeru tipova.

U narednom primjeru, polje kontrola se djelomično preklapa sa poljem ime.

```
union Podatak {
    char ime[50];
    int kontrola;
  } ime;
```

76.4.12 Pointeri i reference

Pointerska varijabla ima opseg vrijednosti koji se sastoji od memorijskih adresa i specijalne vrijednosti, **null**. Vrijednost **null** indicira da se pointer trenutno ne može koristiti da pokazuje na memorijsku ćeliju.

Dvije fundamentalne pointerske operacije su dodjela i dereferenciranje. Dodjela se koristi za postavljanje vrijednosti pointerske varijable na korisnu adresu Ako se pointerske varijable koriste samo za dinamičku memoriju, mehanizam alokacije bilo operatorom bilo potprogramom inicijalizira pointersku varijablu. Ako se koriste za indirektno adresiranje varijabli koje nisu heap dinamičke, mora postojati ugrađeni operator ili potprogram za dobavljanje adrese varijable koja se dodjeljuje pointerskoj varijabli. Dereferenciranje povlači vrijednost na lokaciji predstavljenoj pointerskom vrijednošću. Dereferenciranje može biti eksplicitno ili implicitno C++ koristi eksplicitne operacije preko operatora *.

```
* j = *ptr ; postavlja j na vrijednost koju pokazuje ptr.
```

Pointeri u C i C++ su vrlo fleksibilni ali se moraju oprezno koristiti. Pointeri mogu pokazivati na bilo koju varijablu, bez obzira kada ili gdje je alocirana. Koriste se za dinamičko smiještanje memorije i adresiranje. Moguća je i pointerska aritmetika: uvećavanje ili umanjenje pointera za cjelobrojnu vrijednost i oduzimanje dva pointera. Dereferenciranje je eksplicitno i koristi se simbol *. Primjer pointera u C++:

```
float stuff[100];
float *p;
p = stuff;
//*(p+5) je ekvivalentno s stuff[5] i p[5]
*(p+i) je ekvivalentno s stuff[i] i p[i]
```

C++ uključuje specijalni pointerski tip, referencni tip koji se koristi primarno za formalne parametre. Sljedeći primjer ilustruje reference.

```
int result = 0;
int &ref_result = result;
...
ref_result = 100; //mijenja i result
```

76.4.13 Kompatibilnost tipova

Provjera tipova je aktivnost kojom se osigurava da su operandi operatora kompatibilnih tipova, Kompatibilni tip je onaj koji je legalan za operator ili dopušten pod pravilima jezika da se implicitno konvertuje kompajlerski generisanim kodom u legalan tip. Ova konverzija se zove prinuda (coercion). Ako operatori nisu kompatibilnog tipa dešava se greška tipa. Programski jezik je strogo tipiziran ako se greške tipova odmah prepoznaju. Prednost strogih tipova je što dopušta prepoznavanje varijabli koje rezultuju u greškama tipova.

C i C++ nisu strogo tipizirani, provjera tipova parametara funkcija se može izbjeći, a unije se ne provjeravaju na tipove.

U C **struct, enum** i union imaju ekvivalenciju imenom, a nizovi strukturom. Treba reći da **typedef** ne uvodi novo ime nego samo daje dodatno ime postojećem. U sljedećem primjeru book, anybook i book2 su međusobno ekvivalentnog tipa, a book3 nije. Kada se ukloni komentar, kompajliranje neće proći.

```
#include <iostream>
#include <string>

typedef struct Books {
    char title[50];
    char author[50];
  } Book;

struct knjiga {
    char title[50];
    char author[50];
  } Book3;

int main( ) {  Book book,anybook;
    struct Books book2;
    struct knjiga book3;
    strcpy( book.title, "1984");
    strcpy( book.author, "George␣Orwell");
    book2=book;
    // book3=book;
    return 0;
  }
```

76.5 Aritmetički izrazi

Kao u većini imperativnih programskih jezika, C++ koristi infiksni zapis aritmetičkih izraza, sa prioritetima i operatorima naslijeđenim od Fortrana. Na primjer izraz $\frac{1+\frac{2a}{b+m(x)}}{1+\frac{1}{1+\frac{1}{a}}}$ piše se kao

```
(1+2*a/(b+m(x))/(1+1/(1+1/a))
```

Jezik C++ ne propisuje kojim redoslijedom će se izračunavati pojedini dijelovi izraza koji tvore neki složeniji izraz. Po standardu kompajleri imaju puno pravo da u zavisnosti od okolnosti izaberu redoslijed izračunavanja parametara. Komitet za standardizaciju je smatrao da je nametanje redoslijeda nepotrebno ograničenje za autore kompajlera, s obzirom da postoje računarske arhitekture kod kojih je izračunavanje slijeva nadesno efikasnije od izračunavanja zdesna nalijevo, dok postoje i računarske arhitekture kod kojih vrijedi obrnuto.

76.5.1 Operatori

C++ je jezik bogat operatorima. Nisu svi aritmetički, a neki se u drugim jezicima i ne bi smatrali operatorima, poput indeksa niza, alokacije memorije ili dodjele. Postoji 17 nivoa prioriteta operatora i dvije vrste asocijativnosti, kao što je navedeno u tabeli 76.16 koja sadrži sve operatore.

Pravila asocijativnosti operatora za jezik C++ su predstavljena u tabeli 76.16. Ona su bitna kada se niže više operatora istog prioriteta. Na primjer u izrazu a+b-c, gdje su sabiranje i oduzimanje jednakog prioriteta, postavlja se pitanje da li se prvo izračuna a, pa njemu sabere b, pa od njega oduzme c ili se prvo izračuna c, oduzme od b pa sabere s a. U prvom slučaju govorimo o lijevoj asocijativnosti, a u drugom o desnoj asocijativnosti.

Tablica 76.16: Tabela operatora

Prioritet	Operator	Opis	Asocijativnost
1	::	Razlučivanje opsega	Lijevo
2	a++ a--	Sufiksno uvećanje i umanjenje	
	type() type{}	Promjena tipa funkcionalnim stilom	
	a()	Poziv funkcije	
	a[]	Indeks niza	
	. ->	Pristup članovima niza	
3	++a --a	Prefiksno uvećanje i umanjenje	Desno
	+a -a	Unarni plus i minus	
	! ~	Logički NE i bitni NE	
	(type)	Promjena tipa u C stilu	
	*a	Pokazivana vrijednost	
	&a	Adresa	
	sizeof	Veličina	
	co_await	await-izraz (C ++ 20)	
	new new[]	Dinamička alokacija memorije	
	delete delete[]	Dinamičko oslobađanje memorije	
4	.* ->*	Pointer-na-člana	Lijevo
5	a*b a/b a%b	Množenje ,dijeljenje i ostatak	
6	a+b a-b	Sabiranje i oduzimanje	
7	<< >>	Bitovski pomak ulijevo i udesno	
8	<=>	Trosmjerni operator poređenja (od C ++ 20)	
9	< <=	Za relacijske operatore < i $\leq$	
	> >=	Za relacijske operatore > i $\geq$	
10	== !=	Za relacijske operatore = i $\neq$	
11	&	Bitovsko I	
12	^	Bitovsko XOR (ekskluzivno ili)	
13	|	Bitovsko ILI (uključivo ili)	
14	&&	Logičko I	
15	||	Logičko ILI	
16	a?b:c	Ternarni uslov	Desno
	throw	Operator bacanja	
	co_yield	izraz prinosa (C ++ 20)	
	=	Direktno dodjeljivanje	
	+= -=	Dodjela sa zbirom i razlikom	
	*= /= %=	Dodjela sa proizvodom ,količnikom i ostatkom	
	<<= >>=	Dodjela sa pomakom ulijevo i udesno	
	&= ^= |=	Dodjela sa bitovskim AND ,XOR i OR	
17	,	Zarez	Lijevo

76.5.2 Bočni efekti

Jedna od osobina jezika C i C++ koja ih dosta razlikuje od konvencionalnih programskih efekata je postojanje tzv. izraza sa bočnim efektima (engl. side effects). To su izrazi koji, pored toga što proizvode neku vrijednost, izazivaju i neki prateći (propratni) efekat, najčešće promjenu vrijednosti neke od promjenljivih. Najjednostavniji primjeri izraza sa bočnim efektima su izrazi poput “a = 5”, “a += 2” ili “++a” čija je svrha upravo bočni efekat – izmjena vrijednosti promjenljive “a”, mada svi ovi izrazi imaju i svoj rezultat, koji je značajan ukoliko se oni upotrijebe kao podizrazi u nekom složenijem izrazu. Recimo, u sva tri navedena primjera, vrijednost ovih izraza je upravo nova vrijednost promjenljive “a” nakon izmjene. Po tome se izraz poput “++a” razlikuje od izraza poput “a++” koji također uvećava vrijednost promjenljive “a”, ali je pri tome vrijednost samog izraza stara vrijednost promjenljive “a”, tj. vrijednost ove promjenljive prije izmjene. Ova razlika je vidljiva u složenijim izrazima poput “3 + 2 * (++a)” odnosno “3 + 2 * (a++)”.

U oba ova izraza, kao bočni efekat imamo povećavanje vrijednosti promjenljive “a” za 1, ali će u prvom slučaju za izračunavanje čitavog izraza biti upotrebljena nova vrijednost promjenljive “a”, dok će u drugom slučaju biti upotrebljena stara vrijednost promjenljive “a”. Međutim, kao što ćemo uskoro vidjeti, to ni u kom slučaju ne treba shvatiti da će vrijednost promjenljive “a” nužno biti uvećana tek nakon računanja čitavog izraza.

Izraze sa bočnim efektima treba primjenjivati krajnje oprezno. Naime, specifikacije jezika C i C++ kažu da ukoliko se u nekom izrazu nad nekom promjenljivom primjenjuje bočni efekat (tj. ako izraz mijenja njenu vrijednost), ta se promjenljiva unutar tog izraza smije pojaviti samo na jednom mjestu. U suprotnom, rezultat izvršavanja takvog izraza je nepredvidljiv (preciznije, njegov rezultat zavisi od konkretne izvedbe kompajlera) bez obzira što je sintaksno ispravan (odnosno, kompajler neće na takav izraz prijaviti nikakvu grešku)!

Jezik C++ ne propisuje kojim redoslijedom će se izračunavati pojedini dijelovi izraza koji tvore neki složeniji izraz. Na primjer, u izrazu oblika “ + ” gdje su “ ” i “y ” također izrazi, “ ” i “y ” će sigurno biti izračunati prije nego što započne sabiranje, ali nema nikakvog pravila da li će prvo biti izračunat podizraz “ ” ili podizraz “y ”. Ova neodređenost omogućava kompajleru da sam odabere redoslijed računanja koji je u konkretnom slučaju pogodniji, s obzirom da je nekad bolje prvo izračunati “ ” pa onda “y ”, a nekad obrnuto. Standardom jezika C i C++ je zabranjeno u nekom od stvarnih parametara koristiti promjenljivu nad kojom se u nekom drugom stvarnom parametru istog potprograma obavlja bočni efekat. Ova zabrana nije sintaksne prirode, tako da će kompajler dopustiti ovakve pozive, ali njihov efekat nije predvidljiv.

76.5.3 Preopterećenje operatora

C++omogućava korisničko preopterećenje operatora. Na primjer, mogu se definisati operacije sabiranja i množenja matrica. Operator se onda realizuje pišući potprogram koji obavlja novu operaciju. Na primjer A * B + C * D se može pisati umjesto MatrixAdd(MatrixMult(A, B), MatrixMult(C, D)) Dati primjer pomaže čitljivosti. Međutim, korisnički operatori je mogu i smanjiti, na primjer, ako se definiše da * obavlja oduzimanje matrica. Čitljivost može trpjeti, čak i kada operatori imaju smisla. Na primjer množenje matrice skalarom i množenje matrice matricom se implementiraju u odvojenim potprogramima koje je nekad teško naći. Ovo spada u oblast preklapanje operatora, kada operatoru dodjeljujemo ulogu kakvu želimo.

Sljedeći primjer pokazuje kako se definiše klasa Complex koja ima preopterećeni operator sabiranja, koji se može koristiti između dva objekta.

```
#include<iostream>
using namespace std;

class Complex {
    private:
    int real, imag;
    public:
    Complex(int r = 0, int i =0)  {real = r;   imag = i;}
    Complex operator + (Complex const &obj) {
        Complex res;
        res.real = real + obj.real;
        res.imag = imag + obj.imag;
        return res;
      }
    void print() { cout << real << " + i" << imag << endl; }
  };

int main()
  {
    Complex c1(10, 5), c2(2, 4);
    Complex c3 = c1 + c2; // An example call to "operator+"
    c3.print();
  }
```

76.5.4 Odnosi tipova u izrazima

Dopušteno je miješanje tipova u izrazima, ako su tipovi kompatibilni. Na primjer:

```
float a,b;
int c;
a=b+c;
```

Ovdje je uočljiva implicitna konverzija tipova. Ovaj pristup povećava upisivost i čitljivost, ali smanjuje pouzdanost, jer se smanjuje mogućnost otkrivanja grešaka tipova u kompajleru. Također pored ove konverzije (implicitna, automatska) postoje i druge konverzije tipova u C++.

Automatske konverzije tipova označavaju to da kompajler očekuje podatke jednog tipa ali dobije podatke drugog tipa pa ih kompajler automatski konvertuje, bez da programer eksplicitno to zahtijeva.

Eksplicitne konverzije tipova predstavljaju direktnu i specifičnu notaciju u izvornom kodu koja zahtijeva konverziju. Ovo je potrebno iz razloga što kompajler ne može uvijek izvršiti automatsku konverziju (već može samo u slučajevima kada se radi o sličnim tipovima) ili gdje je kompajler zbunjen i ne zna u koji tip treba pretvoriti. Ovi slučajevi zahtijevaju eksplicitnu instrukciju od programera ili će rezultovati greškom. Eksplicitne konverzije tipova obavljaju se na jedan od šest načina.

1. Tradicionalna eksplicitna konverzija, naslijeđena iz C
2. Eksplicitna konverzija tipova u notaciji oblika funkcije. Razlika ova dva tipa se vidi u primjeru:

```
        short a=2000;
        int b;
        b = (int) a;    // c-like cast notation
        b = int (a);    // functional notation
```

3. **static_cast** namijenjen za konverziju bazne klase u naslijeđenu,
4. **const_cast** namijenjen za zaobilaženje konstantnih objekata,
5. **dynamic_cast** (koristi se za dinamičko dodjeljivanje tipa),

6. **reinterpret_cast** za konverziju pointera bez obzira na njihov tip. Također za dinamičko dodjeljivanje tipa služe **factories** i **template**.

76.6 Osnovne programske strukture

Programske strukture u C++ preuzete su iz njegovog prethodnika C. Poznate su po konciznom zapisu. Aritmetički izrazi mogu biti također naredbe. Izrazi se povezuju kontrolnim strukturama. Uslovne strukture: **if** i else, ponavljajuće strukture (petlje): **while** petlja, **do** - **while** petlja, **for** petlja, naredbe skoka: **break** naredba, **continue** naredba, goto naredba, exit naredba, i selektivna struktura: **switch**.

76.6.1 Naredbe dodjele

Naredbe dodjele u C++ su specijalni slučaj aritmetičkog izraza u kome se nalazi bar jedan operator dodjele. Opšta sintaksa naredbi kojima se dodjeljuje vrijednost varijabli je:

```
izraz1 = izraz2;
```

Operator dodjele u C++ jeziku je znak „=“. C++ je prihvatio skraćeni metod dodjele, tj operatore priračunanja koji omogućavaju kraći zapis aritmetičkih izraza. Sastoje se od znaka jednakosti i odgovarajućeg aritmetičkog operatora. Npr. izraz a=a+8, može se zapisati kao a+=8.

Unarni operatori dodjele u C++ kombinuju inkrementiranje i dekrementiranje s dodjelom. Ovo su primjeri unarnih operatora dodjele:

```
sum = ++count (count uvećan pa dodijeljen u sum)
sum = count++ (count dodijeljen pa uvećan u sum)
count++ (count uvećan)
--count++ (count uvećan pa negiran)
```

U C++, naredbe dodjele imaju rezultat koji se može koristiti kao operand unutar izraza. U sljedećem primjeru se uzme znak funkcijom getchar(), a rezultat (dodijeljen ch) koristi kao uslov za **while** naredbu.

```
while ((ch=getchar())) != EOF {radi() ;}
```

Naredbe dodjele mogu biti u miješanom režimu. Ako odredišna varijabla nije istog tipa kao izraz koji joj se dodjeljuje, postavlja se pitanje kakva konverzija će se obaviti. U C++, svaka vrijednost numeričkog tipa se može dodijeliti varijabli numeričkog tipa .

76.6.2 Naredbe izbora

Ključna riječ **if** se koristi da se izvrši naredba ili blok naredbi samo ako je uslov zadovoljen. Njegova forma je:

```
if (condition)
statement
```

Ovdje je uslov u zagradama (condition). Ako je uslov istinit, naredba će biti izvršena. Ako je neistinit, naredba se ne izvršava i program nastavlja posle uslovne strukture. Na primjer, sljedeći dio koda štampa x je 100 samo ako je vrijednost koja se čuva u promjenljivoj x zaista 100:

```
if (x == 100) cout << "x␣je␣100";
```

Ukoliko želimo da se izvrši više od jedne naredbe u slučaju da je uslov istinit možemo specificirati blok unutar zagrada :

```
if (x == 100)
  {
    cout << "x je ";
    cout << x;
  }
```

Možemo dodatno specificirati šta će se dogoditi ako uslov nije ispunjen korišćenjem ključne riječi **else**. Njena forma korišćenja zajedno sa **if** je:

```
if (condition)
statement1
else
statement2
```

Na primjer:

```
if (x == 100)
cout << "x je 100";
else
cout << "x nije 100";
```

ispisuje na ekranu x je 100 ako x ima vrijednost 100, ali ako nije, ispisuje na ekranu x nije 100.

76.6.3 Višestruki izbor

U jeziku C++ za višestruki izbor se koristi **if-else** ili **switch**. Sljedeći primjer prikazuje njeno korištenje **if-else** ako je tekuća vrijednost smještena u x pozitivna, negativna ili nijedno od toga (odnosno 0):

```
if (x > 0) cout << "x je pozitivno";
else
if (x < 0) cout << "x je negativno";
else cout << "x je 0";
```

Pored **if-else** za višestruki izbor koristimo **switch** naredbu. Cilj **switch** naredbe je da ispita nekoliko mogućih konstantnih vrijednosti određenog izraza. Oblik ove naredbe je

```
switch (expression)
  {
    case constant1: /* grupa naredbi 1 */;
    break;
    case constant2: /* grupa naredbi 2 */;
    break;
    . . .
    default:        /* Naredbe za neispunjene uslove */
  }
```

U ovom primjeru naredba **switch** izvršava izraz i provjerava da li je on ekvivalentan constant1, ako jeste, izvršava se grupa naredbi 1 (group of statements 1) dok se ne naiđe na **break** naredbu. Kada nađe **break** naredbu program skače na kraj **switch** selektivne strukture. Ako izraz nije jednak constant1 ispituje se ekvivalentnost izraza sa constant2.

Ako je ispunjena jednakost, izvršiće se grupa naredbi 2 (group of statements 2) dok se ne naiđe na **break** naredbu, i tada će doći do skoka na kraj **switch** selektivne strukture. Na kraju, ako vrijednost izraza ne odgovara nijednoj od specificiranih konstanti (može se uključiti **case**

labela onoliko koliko vrijednosti treba ispitati), program će izvršavati naredbe koje se nalaze poslije **default**: labele, ako postoji (pošto je opciona).

Sljedeći primjer pokazuje **switch** naredbu i ekvivalentni niz **if** naredbi.

```
switch (x) {
    case 1:
    cout << x << "=" << 1;
    break;
    case 1:
    cout << x << "=" << 2;
    break;
    default:
    cout << "Nepoznato";
  }
```

Ovo su if naredbe koje postižu isti efekat.

```
if (x==1) {
    cout << x << "=" << 1;
  }
else
if (x==2) {
    cout << x << "=" << 2;
  }
else
cout << "Nepoznato";
```

Naredba višestrukog izbora za razliku od jezika kao što su Ada, Ruby ili C# nema obavezan izlazak iz sekvence. Na primjer, da nismo uključili **break** naredbu poslije prve grupe naredbi (case 1), program ne bi automatski skočio na kraj switch selektivnog bloka, već bi se izvršile i ostale naredbe sve dok ne naiđe **break** ili kraj **switch** selektivnog bloka. Iz ovog razloga nije potrebno koristiti zagrade {} za svaki od case-ova, i to može biti korisno za izvršenje istog bloka naredbi za različite vrijednosti ispitivanog izraza. Ako želimo da provjerimo opseg promjenljivih koje nisu konstante, treba da koristimo **if** i **if-else** naredbe.

76.6.4 Sračunati bezuslovni skok

C++ standard ne podržava sračunati **goto**, jer je to vrlo zastarjela programska struktura. Neki C++ kompajleri imaju to kao ekstenziju, uz pomoć pointera na labele. Sljedeći primjer to ilustruje. Ovo je ekstenzija koju ima GNU C++ kompajler i pokazuje da naredba goto kao parametar može imati pointer na labelu, koje su navedene u nizu, čija je vrijednost izračunata.

```
#include <map>
int foo (std::map<int,int>& m, int d, int x) {
static const void* array[] = {&&lab1, &&lab2, &&lab3 };
goto *array[d%3];
lab1: { m[d]= 1;     return 0;     }
lab2: { m[2*d]=x;    return 1;     }
lab3: { m[d+1]= 4*x; return 2;     }
}
```

76.6.5 Brojačke petlje

U C++ funkcionalnost brojačke petlje se postiže naredbom for, koja je u suštini logički kontrolisana petlja, ali se koristi najčešće u primjenama brojačke petlje. Naredba for ima sljedeći opšti oblik:

```
for ([expr_1] ; [expr_2] ; [expr_3]) statement
```

Izračuna se najprije prvi izraz, i dok je drugi izraz zadovoljen, izvršavaju se naredbe u tijelu petlje. Na kraju svake iteracije izvršava se treći izraz. Zbog fleksibilnosti izraza u C, izrazi mogu biti cijele naredbe dodjele, pa čak i sekvence naredbi dodjele razdvojene zarezima. Vrijednost višenaredbenog izraza je vrijednost zadnje naredbe u izrazu. Ako je drugi izraz odsutan, to je beskonačna petlja.

Brojačka petlja u jezicima nastalim iz C nema eksplicitne varijable petlje. Stoga je je legalno za varijablu petlje ili parametre petlje da se mijenjaju u tijelu petlje. Za razliku od nekih jezika gdje se parametri petlje računaju samo jednom, u C++ prvi izraz se računa jednom, a druga dva pri svakoj iteraciji. C++ se razlikuje od C po tome što kontrolni izraz može biti bool tipa, a početni izraz može uključiti definicije varijabli (opseg je od definicije do kraja tijela petlje).

76.6.6 Logički kontrolisana petlja

C i C++ imaju predtest i posttest oblike u kojima kontrolni izrazi mogu biti aritmetički. Petlja sa predtestom radi na način da se izračuna kontrolni izraz i ako je on tačan izvrši tijelo petlje, nakon čega se vraća na testiranje kontrolnog izraza. Njen opšti oblik je sljedeći.

```
while (ctrlexpr)
loopbody
```

Tijelo petlje može biti pojedinačna naredba ili složena naredba u vitičastim zagradama

Petlja sa posttestom izvrši tijelo bar jednom i nakon toga testira uslov. Ako je ispunjen, vraća se na početak petlje. Opšteg je oblika:

```
do
loopbody
while (ctrlexpr)
```

Sljedeći dio koda pokazuje kako izgledaju ove dvije petlje.

```
int i = 0;
while (i < 5) {
    cout << i << "\n";
    i++;
  }

int i = 0;
do {
    cout << i << "\n";
    i++;
  }
while (i < 5);
```

76.6.7 Petlja s korisnički definisanim izlazom

C++ ima bezuslovne izlaze iz petlji za koje se ne navode labele, a to je **break** i bezlabelsku kontrolnu naredbu, **continue**, koja preskače ostatak trenutne iteracije ali ne izlazi iz petlje. Koristeći prekid (**break**) možemo napustiti petlju čak iako uslov za njen kraj nije ispunjen. On može biti korišćen da završi beskonačnu petlju, ili da je natjera na kraj, prije njenog prirodnog kraja. Na primjer, možemo završiti odbrojavanje prije njegovog prirodnog kraja:

```
// primjer za prekid petlje
#include using namespace std;
int main () {
    int n;
    for (n=10; n>0; n--) {
        cout << n << ",␣";
        if (n==3) {
            cout << "prekid!";
            break;
          }
      }
    return 0;
  }
```

Naredba continue uslovljava program da preskoči ostatak petlje u tekućoj iteraciji kao da je dostignut kraj bloka naredbi, uzrokujući da on skoči na početak sljedeće iteracije. Na primjer, preskočićemo broj 5 u našem odbrojavanju:

```
// continue primjer
using namespace std;
#include <iostream>
int main () {
    for (int n=10; n>0; n--) {
        if (n==5) continue;
        cout << n << ",␣";
      }
    cout << "FIRE!\n";
    return 0;
  }
```

76.6.8 Iteracija kroz strukturu podataka

Kako je **for** petlja generalnog oblika, u C++ se može koristiti za korisnički definisan iterator. U narednom primjeru krećemo se kroz ulančanu listu, počevši od korijena liste, sve dok ne dođemo do NULL pokazivača.

```
for (p=root; p!=NULL; p=p->next)
  {
    cout << p->value;
  }
```

76.6.9 Bezuslovni skokovi

Naredba **goto** omogućava da napravimo apsolutni skok u drugu tačku programa. Ovu mogućnost treba koristiti oprezno pošto njeno izvršenje uzrokuje bezuslovan skok ignorišući bilo koji tip ugniježdenih ograničenja. Tačka destinacije je identifikovana labelom, koja se koristi kao argument **goto** naredbe. Labelu čini validni identifikator praćen sa dvije tačke (:). Generalno govoreći, instrukcija nema konkretnu upotrebu u strukturnom ili objektno orjentisanom programiranju, bez obzira što oni koji su ljubitelji low-level programiranja mogu da je pronađu. Primjer:

```
// goto primjer
using namespace std;
#include <iostream>
int main () {
    int n=10;
    loop: cout << n << ",␣";
    n--;
    if (n>0) goto loop;
    cout << "FIRE!\n";
    return 0;
  }
```

76.7 Potprogrami

Potprogrami omogućavaju da se dio programa koji se nalazi na jednom mjestu koristi i na drugim mjestima u programu. Dok definicija potprograma sadrži zaglavlje i tijelo, izvršni kod potprograma, deklaracije potprograma pružaju protokol, ali ne tijelo potprograma . Deklaracije funkcija u C i C++ se često zovu prototipovi. Deklaracije su neophodne u programskim jezicima koji ne dopuštaju reference unaprijed.

76.7.1 Vezivanje i prosljeđivanje parametara potprograma

Unutar deklaracije potprograma navedeni su formalni parametri. Formalni parametar je pomoćna varijabla navedena u zaglavlju potprograma i korištena u njemu. Stvarni parametar predstavlja vrijednost ili adresu koja se koristi tokom naredbe poziva potprograma.

```
void foo(int arg); //arg je formalni parametar
int main()
  {
    int val = 1;
    foo(val);  //val realni parametar
  }
```

Poziciono vezivanje, postoji u većini programskih jezika. Povezivanje stvarnih parametara na formalne je po poziciji: prvi stvarni parametar se poveže na prvi formalni parametar i tako dalje. Ovo je bezbjedno i efikasno ako je lista parametara relativno kratka. Kao što vidimo, u C++ je zastupljeno poziciono povezivanje.

U nekim jezicima (C++, Python, Ruby, Ada, PHP), formalni parametri mogu imati podrazumijevane vrijednosti (ako nisu stvarni parametri proslijeđeni). U C++, podrazumijevani parametri se moraju pojaviti zadnji, jer su parametri vezani za poziciju. U sljedećem primjeru, posljednji parametar ima podrazumijevanu vrijednost

```
float compute_pay(float income, float tax_rate, int exemptions = 1)
```

Poziv se može izvršiti na sljedeći način:

```
pay = compute_pay(20000.0, 0.15);
```

Parametri se mogu proslijediti potprogramu na više načina, zavisno od toga da li se koriste za prosljeđivanje vrijednosti potprogramu ili čitanju iz njega, te načina kako se kopiraju u potprogram. C++ koristi prosljeđivanje po vrijednosti i prosljeđivanje po referenci.

Prosljeđivanje po vrijednosti - kod ovog metoda, vrijednost stvarnog parametra se koristi za inicijalizaciju odgovarajućeg formalnog parametra. Normalno se implementira kopiranjem. Naredni primjer prosljeđuje broj 3 potprogramu dodijeli, ali po povratku, vrijednost varijable br se ne mijenja.

```
#include <iostream>
using namespace std;
void dodijeli(int broj);
int main()
  {
    int br = 3;
    dodijeli(br);
    cout << "Vrijednost␣br␣" << br<< endl;
    return 0;
  }
void dodijeli(int broj)
  {
    broj = 5;
  }
```

Prosljeđivanje po referenci - kod ovog načina prosljeđivanja parametara, prosljeđuje se staza pristupa, što je obično memorijska adresa prosljeđene varijable, elementa niza ili druge memorijske lokacije gdje je podatak. U C (koji u suštini pruža samo prosljeđivanje po vrijednosti) ova funkcionalost zahtijeva upotrebu pointera i operatora adrese. C++ dopušta deklaraciju preko referenci. Verzija prethodnog primjera sa prosljeđivanjem po referenci će promijeniti varijablu br u vrijednost 5.

```
#include <iostream>
using namespace std;
void dodijeli(int & broj);
int main()
  {
    int br = 3;
    dodijeli(br);
    cout << "Vrijednost␣br␣" << br<< endl;
    return 0;
  }
void dodijeli(int & broj)
  {
    broj = 5;
  }
```

U C++ se provjerava da li su proslijeđeni parametri istog tipa kao formalni parametri potprograma. Izuzetak su funkcije deklarisane sa nepoznatim brojem parametara.

```
void a(int m,...);
```

U C i C++ funkcije se ne mogu proslijediti kao parametri. Umjesto toga, mogu se proslijediti pointeri na funkcije i njihovi tipovi mogu uključiti tipove parametara. Tako se parametri mogu provjeravati na tipove.

```
int myfun2 (int, int); // Deklaracija funkcije
int (*pfun2)(int, int);  //pointer na funkcije
pfun2 = myfun2;  // dodjela funkcije pointeru
(*pfun2)(first, second);  // poziv uz dereferenciranje
pfun2(first, second);  // Poziv na drugi način
```

S obzirom da C++ ne dopušta gniježdene potprograme, nema smisla pričati o referentnom okruženju za parametre koji su funkcije.

76.7.2 Opseg varijabli i gniježdenih potprograma

Unutar potprograma možemo deklarisati lokalne varijable, tipove i labele. Opseg važenja varijable je blok programskog koda u kome su definisane uključujući i njegove gniježdene blokove,

a labele važe unutar cijele funkcije. U C++ lokalne varijable su podrazumijevano stek dinamičke. Mogu se opcionalno deklarisati da budu statičke, ključnom riječju **static**. Metode u C++ mogu imati samo stek dinamičke varijable.

Jezici nastali od C i Fortran nisu dopuštali deklaraciju potprograma unutar potprograma. Dakle, C++ ne dozvoljava deklaraciju potprograma unutar drugih potprograma.

76.7.3 Indirektni i rekurzivni poziv potprograma

Što se tiče indirektnih poziva potprograma, u C++ ovu ulogu imaju pokazivači na funkcije. O njima je prethodno bilo govora, pa ih nećemo objašnjavati opet.

U C++ je podržana rekurzija. Sljedeći primjer prikazuje funkciju koja vraća n!.

```
#include <iostream>
using namespace std;

int faktorijel(int x)
  {
    if (x<=1) return 1; //uslov zaustavljanja rekurzije
    else return x*faktorijel(x-1); //korak rekurzije
  }
int main()  {
    int n;
    cin>>n;
    cout<<faktorijel(n)<<endl; //pozivamo funkciju
    return 0;
  }
```

76.7.4 Procedure i funkcije

Postoje dvije kategorije potprograma. Procedure su skup naredbi koje obavljaju određeni račun. Efekat njihovog poziva je kao da je dodana nova naredba. U jezicima koji nemaju odvojene sintaksne mogućnosti za procedure i funkcije (većina jezika nastalih od C) procedure se definišu kao funkcije koje ne vraćaju vrijednosti (tip **void**). Dakle, u C++ se ne razlikuju procedure i funkcije.

76.7.5 Preopterećeni potprogrami

Preopterećeni potprogram ima isto ime kao drugi potprogram u istom referencirajućem okruženju. Na primjer, moguće je imati tri funkcije invert, od kojih jedna kao parametar ima dva cijela broja, druga float, a treća matricu i koje računaju recipročnu vrijednost razlomka, realnog broja ili inverznu matricu. Iako sve tri imaju isto ime, moguće ih je razlikovati jer svaka verzija preopterećenog potprograma ima jedinstven protokol. C++ uključuju preopterećene potprograme, dakle dopušta se korisnicima da pišu više verzija potprograma s istim imenom.

```
#include <iostream>
using namespace std;
void print(int i) {
    cout << "␣Here␣is␣int␣" << i << endl;
  }
void print(double  f) {
    cout << "␣Here␣is␣float␣" << f << endl;
  }
void print(char const *c) {
    cout << "␣Here␣is␣char*␣" << c << endl;
  }
int main() {
    print(10);
    print(10.10);
    print("ten");
    return 0;
  }
```

76.7.6 Generički potprogrami

Generički ili polimorfni potprogram koristi parametre raznih tipova pri pokretanju. Preopterećeni potprogrami su specijalni slučaj polimorfnih programa pružaju ad hoc polimorfizam. Različiti preopterererećeni potprogrami ne moraju se slično ponašati. Objektno orijentisani jezici, podržavaju polimorfizam podtipa, koji znači da metode pisane za tip T, se mogu koristiti i u svim tipovima izvedenim od tipa T.

Generički potprogrami u C++ se zovu predlošci (**template**). Parametar naredbe **template** može biti **class** identifikator ili **typename** identifikator, što je ekvivalentno. Identifikator predstavlja ime tipa. Sljedeći primjeri parametarskog polimorfizma u C++ :

```
template <class Type>
Type max(Type first, Type second) {
    return first > second ? first : second;
  }
```

Sljedeći predložak će imati instance za svaki tip za koji je operator > definisan. Na primjer, ako je tip oba parametra bio **int**, tada nastaje funkcija:

```
int max (int first, int second) {
    return first > second? first : second;
  }
```

Verzije generičkog potprograma se kreiraju implicitno kada je potprogram imenovan u pozivu ili se njegova adresa dobije & operatorom. Tako na primjer ako su postojale varijable i pozivi funkcija kao u sljedećem listingu:

```
int a, b, c;
char d, e, f;
...
c = max(a, b);
f = max(d, e);
```

U gornjem primjeru nastaju dvije kopije potprograma max, jedan za poređenje dvije int i jedan za poređenje dvije char varijable.

76.7.7 Korutine

Korutina je je potprogram s više ulaza. One se razlikuju od običnih potprograma koji rade na principu da pozivalac čeka da se cijeli potprogram završi. Kod korutina pozivalac i pozvana ko-

rutina moraju biti na jednakom nivou. Korutine su uvedene tek u C++ 20. Funkcija je korutina ako koristi neke od sljedećih ključnih riječi:

1. operator co_await koji zaustavlja izvršenje korutine do njenog nastavka
2. operator co_yield koji zaustavlja izvršenje i vraća vrijednost
3. ključnu riječ co_return koja završava korutinu i vraća vrijednost

Sljedeća funkcija se može pozvati iz petlje i vraća narednu vrijednost (izvršava se bez kraja) svaki put kada dođe do operatora yield.

```
generator<int> iota(int n = 0) {
    while(true)
    co_yield n++;
  }
```

76.8 Apstraktni tipovi podataka i jedinice enkapsulacije

C++ pruža dvije enkapsulacijske konstrukcije.

Jedna je preuzeta iz C: mogu se definisati datoteke s zaglavljem i kodom C.

Druga enkapsulacijska konstrukcija su klase. Kada se one koriste za enkapsulaciju, klasa se deklariše u jednoj datoteci, kao interfejs (prototip), dok su definicije članova u posebnoj datoteci. C++ podržava apstraktne tipove podataka kroz klase.

U velikim projektima C++ koristi imenske prostore za razlučivanje konfliktnih imena.

76.8.1 Klase

Klasa predstavlja apstrakciju koja definira zajednička svojstva i zajedničke postupke koji su svojstveni svim objektima nekog tipa. C++ je proširenje jezika C klasama i objektima. Ovo proširenje je bazirano na C **struct** tipu i Simula 67 klasama. U C++ klasa je jedinica enkapsulacije. C++ klase su tipovi, za razliku od Ada paketa koji definišu više različitih tipova. Elementima C++ klase može pristupati samo kroz instance klase. Stoga se apstraktni tipovi podataka definišu baš kao takvi.

Podaci u C++ klasama se zovu podatkovni članovi, dok se operacije nad njima zovu članske funkcije. Podatkovni članovi i članske funkcije se javljaju u dvije verzije: zajedničke za klasu i vezane za instancu. Svi primjerci (instance) klase dijele jednu kopiju funkcija članice, ali svaki primjerak klase ima vlastitu kopiju podatkovnih članova. Instance mogu biti statičke, stek dinamičke, ili heap dinamičke. Statičkim i stek dinamičkim instancama pristupa se direktno kroz vrijednost varijabli, dok se heap dinamičkim pristupa preko pointera. Heap dinamičke instance se kreiraju **new** operatorom a uništavaju **delete** operatorom.

76.8.2 Konstruktori i destruktori

Dvije posebne vrste članskih funkcija su konstruktori i destruktori. Konstruktori su funkcije koje inicijaliziraju podatke instance. Imaju isto ime kao ime klase, ali ih može biti više kopija s različitim vrstama parametara. Implicitno su pozvani kada se kreira instanca, a mogu se i eksplicitno pozvati. Mogu alocirati memoriju ako je dio objekta heap-dinamički. Destruktori su funkcije koje se pozivaju nakon uništenja instance, obično za oslobađanje prostora na heap. Implicitno su pozvani na kraju života objekta, a mogu se i eksplicitno pozvati. Ime destruktora je ime klase ispred koga je tilda (~).

76.8.3 Format deklaracije klase

Specifikacija tipa može a i ne mora biti fizički odvojena od implementacije, jer C++ dopušta dva različita stila deklaracije klasa.

U prvoj verziji, C++ dopušta da se tijelo članskih funkcija navodi u samoj njenoj definiciji. Primjer definicije takve klase u jeziku C++, koja sadrži tri privatna člana , tri javne članske funkcije, konstruktor i destruktor implementacija klase izgleda ovako:

```
#define STACK_MAX 100
class Stack {
    private:
        int        data[STACK_MAX];
        int        size;
    public:
        Stack() {       // Constructor
            size = 0;
        }
        ~Stack() { }    // Destructor
        int Top() {
            if (size == 0) {
                fprintf(stderr, "Error: stack empty\n");
                return -1;
            }
            return data[size-1];
        }
        void Push(int d) {
            if (size < STACK_MAX)
                data[size++] = d;
            else
                fprintf(stderr, "Error: stack full\n");
        }
        void Pop() {
            if (size == 0)
                fprintf(stderr, "Error: stack empty\n");
            else
                size--;
        }
};

void main () {
   int topOne;
   Stack stk; // Kreiranje
   stk.push(80);
   topOne=stk.top
   stk.pop();
}
```

U prethodnom primjeru sakrivanje informacija je prisutno pri kompajliranju korisnika Stack klase, ali ne i na nivou analize izvornog koda.

U drugoj varijanti, C++ dopušta da se klase pišu iz dva dijela, zaglavlja i tijela, koji se tada čuvaju u odvojenim datotekama. Tako je zaglavlje Stack klase opisano u sljedećem kodu.

```
  // Stack.h - the header file for the Stack class
  #define STACK_MAX 100
  class Stack {
      private:
      int        data[STACK_MAX];
      int        size;
      public:
      Stack() { size = 0; }    // Constructor
      ~Stack() { }             // Destructor
      int Top();
      void Push(int d);
      void Pop();
    };
```

Implementacija klase je navedena u odvojenoj datoteci, djelomično prikazana u sljedećem

listingu.

```
int Stack::Top()
{
    if (size == 0) {
        fprintf(stderr, "Error:␣stack␣empty\n");
        return -1;
    }
    return data[size-1];
}

void Stack::Push(int d)
{
    if (size < STACK_MAX)
        data[size++] = d;
    else
        fprintf(stderr, "Error:␣stack␣full\n");
}

void Stack::Pop()
{
    if (size == 0)
        fprintf(stderr, "Error:␣stack␣empty\n");
    else
        size--;
}

int Stack::Top()
{
    if (size == 0) {
        fprintf(stderr, "Error:␣stack␣empty\n");
        return -1;
    }
    return data[size-1];
}

void Stack::Push(int d)
{
    if (size < STACK_MAX)
        data[size++] = d;
    else
        fprintf(stderr, "Error:␣stack␣full\n");
}

void Stack::Pop()
{
    if (size == 0)
        fprintf(stderr, "Error:␣stack␣empty\n");
    else
        size--;
}
```

76.8.4 Friend funkcije

U C++ se dešava nedostatak generalnije konstrukcije enkapsulacije za operacije koje djeluju nad objektima iz različitih klasa ali koje prirodno ne pripadaju ni jednoj od njih. Na primjer ako postoji klasa vektora i klasa matrica, a potrebno je definisati funkciju za množenje vektora matricom, ta funkcija bi morala pristupati internim članovima obije klase, a ne bi se mogla definisati ni u jednoj od njih. Rješenje ovog problema predstavljaju **friend** funkcije. One pružaju pristupu privatnim elementima klase tako što se funkcija deklarisana izvan klase ključnom riječju **friend** uključi u klasu,

```
class Matrix; //** A class declaration
class Vector {
    friend Vector multiply(const Matrix&, const Vector&);
    ...
  }; class Matrix { //** The class definition
    friend Vector multiply(const Matrix&, const Vector&);
    ...
  };
//** The function that uses both Matrix and Vector objects
Vector multiply(const Matrix& m1, const Vector& v1) {
    ...
  }
```

76.8.5 Parametrizovane klase

Nekada je potrebno parametrizirati apstraktne tipove Parameterizirani ADT dopuštaju dizajn ADT koji mogu smjestiti elemente bilo kog tipa, na primjer u koga se može smjestiti po potrebi stek stringova, cijelih brojeva ili objekata. Problem postoji samo kod jezika s statičkim povezivanjem tipova, jezici kod kojih se tip dinamički određuje (Ruby, Python, JavaScript) to postižu automatski.

Jezici s statičkim povezivanjem tipova među kojima je i C++ pružaju podršku za parametrizirane ADTs.

C++ pruža parametrizirane ADT kroz konstruktore s parametrima i generičke klase kroz predloške. U primjeru sa stekom, Konstruktori s parametrima rješavaju problem fiksne veličine.

```
class Stack {
    private: Type *stack_ptr;
    const int max_len;
    int top;
    public:
    Stack(int size) {
        stk_ptr = new int [size];
        max_len = size -1;
        top = -1;
      } ;
    ...
  }
Stack stk(100);
```

Predlošcima se može realizovati tip stack sa proizvoljnim tipom podataka koji se smješta u njega.

```
<class Type> class Stack {
    private: Type *stack_ptr;
    const int max_len;
    int top;
    public:
    Stack() {
        stk_ptr = new Type [100];
        max_len = 99;
        top = -1;
      }
  } ;
```

76.8.6 Kontrola pristupa u klasama

Skrivanje informacija je postignuto kroz sekcije unutar deklaracije klase. Ključna riječ **private** određuje skrivene entitete, koji se mogu koristiti samo unutar instance klase. Entiteti koji su dostupni klijentima opisani su uz ključnu riječ **public**. Entiteti kojima se može pristupati samo u naslijeđenim klasama su definisani unutar sekcije označene ključnom riječju **protected**. U izvedenoj klasi možemo koristiti **public** i **protected** članove bazne klase. Međutim, nemamo pristup privatnim dijelovima klase, kao u sljedećem primjeru.

```
Stack s(50);
cout <<s.top; // ovo izaziva grešku.
```

76.8.7 Paketi

U C++ se programi mogu pisati tako da se razdvoji zaglavlje datoteke i programski kod. Zaglavlje može sadržati prototipe funkcija, deklaracije varijabli, korisničkih tipova i klasa. Ono se napiše u jednoj datoteci izvornog koda. Druga datoteka je sa programskim kodom i ona može koristiti direktivu #include na početku, čiji je parametar ime datoteke sa zaglavljem, čime se postiže kao da je datoteka sa zaglavljem ukucana na tom mjestu datoteke sa izvornim kodom. Datoteka sa zaglavljem se može uključiti u različite datoteke sa izvornim kodom. Pošto se povezivanje modula obavlja nad objektnim kodom, moguće je podmetnuti zaglavlje nekompatibilne implementacije, što može rezultovati vrlo ozbiljnim greškama.

76.8.8 Imenske enkapsulacije

U velikim projektima spajaju se dijelovi koda iz raznih izvora i velika je vjerovatnoća da će se pojaviti više globalnih varijabli, klasa ili funkcija koji imaju isto ime i ostale parametre koji ih čine jedinstvenim. Tada se mora uvesti način da se izabere koja od navedenih varijabli, klasa ili funkcija sa istim imenom će se koristiti u datom trenutku. C++ pruža rješenje ovog problema primjenom imenskih prostora.

Imenski prostori omogućavaju grupisanje klasa, varijabli, objekata i funkcija pod zajedničkim imenom. Na taj način, globalni opseg se dijeli na više podopsega, a svaki ima svoje ime. Format imenskog prostora izgleda ovako.

```
namespace ime
  {
    sadrzaj
  }
```

Sadržaj je skup klasa, varijabli, objekata i funkcija uključenih u imenski prostor. U sljedećem primjeru varijable a i b su normalne varijable deklarisane unutar imenskog prostora vv.

```
namespace vv
  {
    int a, b;
  }
```

Za pristup varijablama izvan imenskog prostora treba se koristiti operator opsega **::** , kao u sljedećem primjeru .

```
vv::a=18;
vv::b =20;
```

Ključna riječ **using** se koristi za uvođenje imena iz imenskog prostora u na dato mjesto deklaracije. Sljedeći primjer razlučuje imena x i y iz dva različita imenska prostora.

```
#include <iostream>
using namespace std;
namespace first
  {
    int x = 5;
    int y = 10;
  }
namespace second
  {
    double x = 3.1416;
    double y = 2.7183;
  }
int main () {
    using first::x;
    using second::y;
    cout << x << y << first::y << second::x << endl;
    return 0;
  }
```

Rezultat je 5 2.7183 10 3.1416

Ključna riječ **using** u obliku **using namespace** se može koristiti i kao direktiva za uključivanje cijelog imenskog prostora. U prethodnom primjeru se objektu **cout** može pristupiti bez navođenja imenskog prostora, zato što je uključen imenski prostor std.

Imenski prostor veži na nivou bloka ili cijele datoteke sa izvornim kodom. Moguće je unutar izvornog koda programa prelaziti sa jednog imenskog prostora na drugi. Sljedeći primjer ispisaće 5 i 3.1416.

```
// using namespace example
#include <iostream>
using namespace std;
namespace first
{
  int x = 5;
}
namespace second
{
  double x = 3.1416;
}
int main () {
  {
    using namespace first;
    cout << x << endl;
  }
  {
    using namespace second;
    cout << x << endl;
  }
  return 0;
}
```

Može se definisati dodatno ime za postojeće imenske prostore.

```
namespace novoime = trenutnoime;
```

Najvažniji imenski prostor je std, pošto je on vezan za standardnu biblioteku. Iz tog razloga se uključuje u gotovo svakom programu.

76.8.9 Biblioteke

Kao jedan od najpopularnijih programskih jezika. C++ omogućava veliki broj programskih biblioteka, koje se koriste kao skupovi funkcija ili klasa. Prije upotrebe funkcija i klasa u bibli-

oteci treba uključiti imenski prostor naredbom **using namespace**, a zatim direktivom **#include** uključiti odgovarajuće zaglavlje.

Uz C++ kompajlere se obično isporučuju standardna biblioteka i dodatne biblioteke specifične za konkretni kompajler i operativni sistem. Spomenućemo skupove funkcija i klasa koje pruža standardna biblioteka.

Uz dodatak osnovnim tipovima podataka, izvedeno je više dodatnih tipova podataka (tablica 76.17). Neki od njih se nazivaju kontejneri, koji predstavljaju strukture podataka u koje se ugrađuju elementarni tipovi, objekti ili strukture.

Tablica 76.17: Izvedeni tipovi podataka

Zaglavlje	Namjena
<array>	Niz fiksne veličine
<bitset>	Niz bitova
<deque>	Dvostrani red
<forward_list>	Jednostruko spregnuta lista
<list>	Dvostruko spregnuta lista
<map>	Asocijativni niz
<queue>	Jednostrani red
<set>	Sadržalac skupova
<stack>	Stek
<unordered_map>	Heš tabele
<unordered_set>	Heš tabele
<vector>	Dinamički niz
<string>	Klasa stringova
<regex>	Klasa regularnih izraza
<tuple>	n-torke
<utility>	Uređeni parovi
<complex>	Kompleksni brojevi
<valarray>	Nizovi numeričkih vrijednosti
<iterator>	Kretanje kroz strukturu podataka

Ulaz/izlaz u standardnoj C++ biblioteci i dalje je ograničen na datoteke i konzolni pristup.

Tablica 76.18: Ulazno/izlazne operacije

<fstream>	Datotečni ulaz/izlaz
<iomanip>	Format brojeva za prikaz
<ios>	Ulazno/izlazni tokovi
<iosfwd>	Predlošci klasa za ulaz/izaz
<iostream>	Osnovni ulaz izlaz
<istream>	Strujanje podataka ulaza
<ostream>	Strujanje podataka izlaza
<sstream>	Strujanje podataka kroz stringove
<streambuf>	Čitanje i pisanje sekvenci znakova

Različite korisne rutine postaju moguće uključivanjem zaglavlja iz tabele 76.19.

Tablica 76.19: Razne funkcije

<algoritm>	Razni algoritmi
<random>	Slučajni brojevi
<numeric>	Razne numeričke operacije
<chrono>	Vremenske funkcije
<memory>	upravljanje memorijom
<locale>	informacije o lokalnom jeziku
<codecvt>	kodiranja znakova

Od C++ 11 uključene su klase koje podržavju asinhrone događaje i druge paradigme.

Tablica 76.20: Podrška asinhronim događajima

<exception>	Upravljanje izuzecima
<stdexcept>	Klase standardnih izuzetaka
<limits>	Ograničenja osnovnih tipova
<new>	Operatori **new** i **delete**
<typeinfo>	Prepoznavanje tipova u toku izvršavanja
<thread>	Podrška za niti
<mutex>	Mehanizmi uzajamnog isključivanja
<condition_variable>	Sinhronizacijske uslovne varijable
<future>	Čekanje da se varijabla asinhrono dodijeli
<functional>	Funkcijski objekti

Biblioteka za C++ uključuje i standardnu biblioteku za C, ali imaju verzije ugrađene u imenski prostor **std**. Datoteke sa zaglavljem u C se uključuju u C++ program sa imenom koje nema nastavak ".h", a ispred tako dobivenog imena se dodaje slovo c. Tako umjesto

```
#include <stdio.h>
#include <math.h>
#include <string.h>
```

u C++ programima se piše

```
#include <cstdio>
#include <cmath>
#include <cstring>
```

76.9 Podrška objektno-orijentisanom programiranju

Objektno-orijentisano programiranje (OOP) je izrazito popularna paradigma u programskim jezicima, jer je praktična za veće softverske projekte u kojima je moguće koristiti isti kod za različite namjene. Programski jezici koji podržavaju objektno orijentisano programiranje se razlikuju po svom nastanku i načinu kako je ono dodano u taj jezik. C++ podržava proceduralno i objektno orijentisano programiranje.

76.9.1 Nasljeđivanje

Nasljeđivanje (engl. inheritance) je metodologija koja omogućava definiranje klasa ili objekata koje preuzimaju većinu svojstava nekih već postojećih klasa ili objekata i koje se mogu koristiti u svim kontekstima u kojima se mogu koristiti te postojeće klase. Na primjer, ukoliko definišemo da je klasa "MotornoVozilo" naslijeđena iz klase "Vozilo", tada klasa "MotornoVozilo"

automatski preuzima sve atribute i metode koje je posjedovala i klasa "Vozilo", pri čemu je moguće u klasi "MotornoVozilo" dodati nove atribute i metode koje klasa "Vozilo" nije posjedovala, kao i promijeniti definiciju neke od naslijeđenih metoda.

```
using namespace std;
#include <iostream>
//Bazna klasa
class Vozilo
{
    public:
      int tezina;
      void printtezina() {cout << tezina;}
};
// Izvedena klasa
class MotornoVozilo : public Vozilo
{
    public:
      double potrosnja;
      void printpotrosnja() {cout << potrosnja;}
};

int main()
{
  MotornoVozilo obj1;
  obj1.tezina = 800;
  obj1.potrosnja = 9.1;
  obj1.printtezina();
  obj1.printpotrosnja();
  return 0;
}
```

76.9.2 Odnos klasa i objekata prema ostalim tipovima podataka

Za razliku od jezika Smalltalk, Ruby, Eifell kod kojih su svi tipovi podataka objekti, u C++ to nije slučaj. C++ ima dva sistema tipova, sistem izveden od elementarnih tipova i sistem izveden od klasa. Mana tog pristupa je glomaznost jezika, ali (za razliku od npr. Objective C) nije narušena konzistentnost između upotrebe varijabli jedne i druge vrste tipa.

Elementarni tipovi nisu klase i ne postoji bazna klasa. Moguće je imati više hijerarhija klasa u jednom projektu.

Izvedena klasa je podtip ako ima "je" odnos s roditeljskom klasom, tj. da sve operacije roditeljske klase budu moguće i u izvedenoj klasi. U većini jezika podklasa može samo dodavati varijable i metode i preklapati naslijeđene metode na kompatibilan način. Nekada nam to ne odgovara. Primjer je klasa Kvadrat koja je logično podklasa klase Pravougaonik. Ali, kako ne smijemo promijeniti dvije stranice kvadrata (jer tada on prestaje biti kvadrat), ne možemo imati iste operacije u podklasi kao u roditeljskoj. C++ ima dvije vrste nasljeđivanja, prvu u kojoj podklase jesu podtipovi, takozvano **public** izvođenje, i drugu u kojoj nisu, **private** izvođenje. S privatnim nasljeđivanjem, javni i zaštićeni član roditeljske klase postaju privatni članovi izvedene klase. To znači da metode osnovne klase ne postaju javno dostupni u izvedenoj klasi. Međutim, mogu se koristiti unutar funkcija člana izvedene klase. U narednom primjeru imamo klasu Dijete koja je privatno izvedena od klase Osoba. Pošto je dijete logično osoba, ali nema kreditnu karticu, odlučilo se za privatno izvođenje kako bi odgovarajuća metoda bila isključena. S druge strane, objektima klasa Muskarac i Zena dostupne su metode klase Osoba i nove metode

```
class Osoba {
   public:
   registrujkreditnukarticu();
};

```

```
class Dijete:private Osoba {
   public:
   registrujskolu() {}
};

class Zena:public Osoba {
   public:
   registrujporodjaj() {}
};

class Muskarac:public Osoba {
   public:
   registrujvojnirok() {}
};

int main()
{
   Muskarac Ivo;
   Zena Vesna;
   Dijete Tom;
   Tom.registrujskolu (); // Dopusteno
   Tom.registrujkreditnukarticu (); // Nije dopusteno
   Vesna.registrujkreditnukarticu (); // Dopusteno
   Vesna.registrujporodjaj (); // Dopusteno
   return 0;
}
```

76.9.3 Polimorfizam i načini povezivanja

Polimorfizam (engl. polymorphism), ili jednostavno ”moć poprimanja više oblika” je osobina po kojoj metoda ima parametre različitih tipova. Npr. dodaj() metoda može imati brojčane parametre ili tekstualne, ovisno o upotrebi. U sljedećem primjeru, klasa A ima metodu koja radi jednu aktivnost, a klasa B metodu koja radi drugu aktivnost.

```
class A {
public:
  virtual void method() = {
    cout << "Poruka␣1" << endl;
  }
};

class B : public A {
public:
  void method() {
    cout << "Poruka␣2" << endl;
  }
};

int main()
{
  A *p = new B; //Iako je pokazivac tipa A, zbog polimorfizma mu je moguce
      pridruziti B
  p->method(); //Na ekranu ce se ispisati Poruka 2.
  return 0;
}
```

Kada se naslijedi klasa, metoda preklopi drugom istoimenom, i dodijeli objekt naslijeđene klase objektu roditeljske klase otvara pitanje koju od te dvije istoimene metode pozvati nad objektom roditeljske klase. Kod statičkog povezivanja pozvaće se metoda iz roditeljske klase, a kod dinamičkog iz naslijeđene klase.

U C++ dinamičko povezivanje je opcionalno. Metod se može definisati kao **virtual**, što znači da se može pozivati polimorfnim varijablama i dinamički povezati s porukama. Neka su definisane sljedeće klase:

```
class Shape {
public:
  virtual void draw() = 0;
   /*...*/
};

class Circle : public Shape {
public:
 void draw() { /*. . .*/ }
 /* ... */
};

class Rectangle : public Shape {
public:
  void draw() {/* . . .*/ }
  void fill() { /*. . .*/ }
};

 class Square : public Rectangle {
public:
  void draw() {/* . . .*/ } ...
 };
```

Sljedeća upotreba ovih klasa ilustruje poziv i sa dinamički povezanim i sa statički povezanim metodama.

```
  Square* sq = new Square;
  Rectangle* rect = new Rectangle;
  Shape* ptr_shape;
  ptr_shape = sq; // ptr_shape pokazuje na Square objekt
  ptr_shape->draw(); //Dinamički vezan draw u Square klasi
  rect->fill(); //Statički vezan fill u Rectangle klasi
```

Virtualna metoda čija je definicija postavljena na nulu (draw u Shape) je čisto virtualna funkcija. Stoga klasa **Shape** ima smisla postojati jedino kao bazna klasa iz koje će se dalje izvoditi druge klase

76.9.4 Apstraktne klase i interfejsi

Klasa koja ima barem jednu virtualnu funkciju se zove apstraktna klasa i od ovih klasa se ne može praviti instanca. Apstraktne klase se definišu kao takve posebnim pravilima. U objektno orijentiranom programiranju apstraktna klasa je klasa koja ne može imati objekte, tj. instance. Njena osnovna svrha je da bude roditelj drugim, konkretnim klasama. Iz tog razloga ne može imati sve metode implementirane i treba imati barem jednu apstraktnu metodu, tj. čisto virtualnu metodu u C++u.

U C++ ne postoji pojam interfesa (interface) iz Jave, umjesto njega koristi se apstraktna klasa u kojoj su sve metode čiste virtuelne, a koja nema podataka članova.

76.9.5 Alokacija objekata

C++ dopušta tri načina alokacije objekata. Objekti mogu biti statički, stek dinamički i heap dinamički. Ako su heap dinamički potrebna je eksplicitna dealokacija objekata koristeći delete operator. Definicije klasa mogu uključivati konstruktore i destruktore koji se implicitno pozivaju prilikom kreiranja ili uništavanja objekata. U sljedećem primjeru kreiraju se četiri objekta

klase Matrica. Objekt Mat1 se kreira statički, kao globalni, u sekciji podataka. Pri tome se pozove konstruktor sa dva parametra. Objekt Mat4 se kreira statički, kao lokalni, također u sekciji podataka. Pri tome se pozove konstruktor bez parametara. Objekt Mat2 se kreira dinamički na steku, sa konstruktorom od dva parametra. On se briše pri napuštanju funkcije alociraj. Objekt Mat5 se kreira dinamički na heapu, ali se mora eksplicitno alocirati i dealocirati odgovarajućim operatorom. Pri tome mu se mora pristupati preko pointera.

```
#include "klasa.hpp"
Matrica Mat1(100,20);

void alociraj() {
    Matrica Mat2(40,10);
    static Matrica Mat4;
    Matrica * Mat5 = new Matrica(8,10);
    Mat1.dodijeliElement(12,5,10);
    Mat2.dodijeliElement(2,5,8);
    Mat5->dodijeliElement(3,5,8);
    delete Mat5;
  }

void main() {
    alociraj();
  }
```

Ako su objekti stek dinamički postoji problem s odsijecanjem pri dodjeli podtipa roditelju. U sljedećem primjeru u C++, kopiranje varijable b1 u varijablu a1 će odsjeći dodatne elemente koje je uvela klasa B.

```
class A { int x; }
class B : A { int y ;}
void test() {
    A a1;
    B b1;
    a1=b1 ;
  }
```

76.9.6 Inicijalizacija objekata

Nakon alociranja objekta, poziva se jedan od konstruktora čije tijelo obično sadrži naredbe za dodjelu vrijednosti članovima objekta.

```
class Kompleksni {
    double re, im;
    public:
    Kompleksni() { re = im = 0; }
    Kompleksni(double x) { re = x; im = 0; }
    Kompleksni(double r, double i) { re = r; im = i; }
    void Postavi(double r, double i) { re = r; im = i; }
    void Ispisi() const { std::cout << "(" << re << "," << im << ")"; }
    double DajRealni() const { return re; }
    double DajImaginarni() const { return im; }
    friend Kompleksni ZbirKompleksnih(const Kompleksni &a, const Kompleksni &
        b);
  };
```

U primjeru klase Kompleksni, implementirana su 3 konstruktora. Ukoliko u kodu napišemo:

```
Kompleksni a;
Kompleksni b(3.2,6.8);
std::cout << a.DajRealni << "␣" << a.DajImaginarni(); std::cout << b.
    DajRealni << "␣" << b.DajImaginarni();
```

Na ekranu će se ispisati 0 , 0, 3.2 i 6.8. Prilikom deklaracije objekta "a" automatski je nad njim primijenjen konstruktor bez parametara koji je izvršio inicijalizaciju atributa "re" i "im" na nule (u skladu sa naredbom unutar tijela konstruktora).

Kada se konstruktor koristi za inicijalizaciju drugih članova, ti drugi članovi se mogu inicijalizirati direktno, bez ubacivanja u naredbe tijela. To se radi ubacivanjem prije tijela konstruktora dvotačke i liste inicijalizatora za članove klase. Na primjer, klasa ima sljedeću deklaraciju:

```
class Rectangle {
    int width,height;
    public:
    Rectangle(int,int);
    int area() {return width*height;}
  };
```

Konstruktor za ovu klasu se može definisati na uobičajen način.

```
Rectangle::Rectangle (int x, int y) { width=x; height=y; }
```

Ali se inicijalizacija može obaviti i prije tijela konstruktora.

```
Rectangle::Rectangle (int x, int y) : width(x), height(y) { }
```

Ovdje je tijelo konstruktora ostalo prazno, jer je sva njegova funkcionalnost (inicijalizacija članova objekta) obavljena ranije.

76.9.7 Višestruko i ponovljeno nasljeđivanje

Višestruko nasljeđivanje dopušta da klasa nasljeđuje podatke i metode iz dvije ili više klasa koje nisu u istom stablu klasa. To omogućava prirodnije modeliranje nekih podataka. Npr. klasa KinderJaje može biti istovremeno nasljednik klase Čokolada i klase Igračka. Ipak postoje mane višestrukog nasljeđivanja. Višestruko nasljeđivanje povećava jezičku i implementacijsku kompleksnost zbog sukoba imena. Na primjer ako su obije klase Čokolada i Igračka definisale metodu PostaviBoju, nejasno je od koje klase se preuzima ova metoda u klasi KinderJaje.

Sljedeći primjer deklariše tri klase, od kojih treća nasljeđuje istovremeno ostale dvije. Kako klasa C nasljeđuje prvo klasu B, pa A, pri kreiranju objekta klase C, pozvaće se konstruktor B, pa A, pa C. Klasa C će imati sve dostupne metode i varijable i klase A i klase B.

```
#include<iostream>
using namespace std;

class A
{
public:
  int z;
  void m1(int a) {cout << a;}
  A()  { cout << "A's␣constructor␣called" << endl; }
};

class B
{
public:
  char z;
  void m2(int b) {cout << b;}
```

```
  B()  { cout << "B's␣constructor␣called" << endl; }
};

class C: public B, public A  // Note the order
{
public:
  C()  { cout << "C's␣constructor␣called" << endl; }
};

int main()
{
    C c;
    c.m1();
    c.m2();
    cout <<c.B::z;
    return 0;
}
```

Ako postoje dva naslijeđena člana s istim imenom (što se zove ponovljeno nasljeđivanje) može im se prići operatorom razlučivanja opsega :: kao u prethodnom primjeru sa članskom varijablom z.

76.10 Gniježdene klase

Ugniježdena klasa je klasa koja je deklarisana unutar druge klase. C++ podržava rad sa ugniježdenim klasama. Program u nastavku će se kopajlirati bez ikakve greške. Pravilo je da ugniježdena klasa može pristupati privatnim atributima obuhvatajuće klase, ali obrnuto ne važi.

```
using namespace std;
class Enclosing {
    int x;

    class Nested {
        int y;
        void NestedFun(Enclosing *e) {
            cout<<e->x;  // radi bez greške, ugniježdena klasa može
                pristupiti
            // privatnim atributima obuhvatajuće klase
          }
      };
  };

int main(){ }
```

Sljedeći program će imati grešaka pri kompajliranju.

```
#include<iostream>
using namespace std;
class Enclosing {
    int x;

    class Nested {
        int y;
      }; // deklaracija Nested klase završava ovdje

    void EnclosingFun(Nested *n) {
        cout<<n->y;  // Greška: y je privatni atribut
      }
  };

int main(){}
```

76.11 Podrška asinhronim događajima

76.11.1 Konkurentnost

Jezgro jezika nema podršku za konkurentnost. Međutim, standardna biblioteka uključuje klasu thread koja to omogućava. Sljedeći primjer pokreće dvije programske niti paralelno.

```
// thread example
#include <iostream>       // std::cout
#include <thread>         // std::thread
void foo()
{
  // do stuff...
}
void bar(int x)
{
  // do stuff...
}
int main()
{
  std::thread first (foo);     // spawn new thread that calls foo()
  std::thread second (bar,0);  // spawn new thread that calls bar(0)
  std::cout << "main,␣foo␣and␣bar␣now␣execute␣concurrently...\n";
  // synchronize threads:
  first.join();                // pauses until first finishes
  second.join();               // pauses until second finishes
  std::cout << "foo␣and␣bar␣completed.\n";
  return 0;
}
```

Sinhronizacija niti se postiže primjenom klase **mutex**, čija metoda **lock** služi za zaključavanje kritične sekcije, dok je metoda **unlock** otključava. Sljedeći primjer pokazuje upotrebu ove klase.

```
#include <iostream>       // std::cout
#include <thread>         // std::thread
#include <mutex>          // std::mutex
std::mutex mtx;           // mutex for critical section
void print_block (int n, char c) {
  mtx.lock();
  for (int i=0; i<n; ++i) { std::cout << c; }
  std::cout << '\n';
  mtx.unlock();
}
int main ()
{
  std::thread th1 (print_block,50,'*');
  std::thread th2 (print_block,50,'$');
  th1.join();
  th2.join();
  return 0;
}
```

76.11.2 Podrška izuzecima

Izuzeci su preporučeni način reagiranja funkcije na nepredviđene okolnosti, npr. na argumente za koje funkcija nije definirana. U slučaju da funkcija ustanovi da ne može odraditi svoj posao, ili da ne može vratiti nikakvu smislenu vrijednost (npr. stoga što nije definirana za navedene argumente), funkcija umjesto da vrati vrijednost treba da baci izuzetak. Obratimo pažnju na terminologiju: vrijednosti funkcije se vraćaju iz funkcije, a izuzeci se bacaju. Za bacanje izuzetaka koristi se naredba "**throw**", koja ima sličnu sintaksu kao naredba "**return**": iza

nje slijedi izraz koji identificira izuzetak koji se baca (kasnije ćemo vidjeti smisao ove identifikacije). Uzmimo kao primjer funkciju za računanje faktorijela (povratni tip ćemo deklarirati kao "**long long int**" da maksimalno proširimo opseg cjelobrojnih vrijednosti za koje se može dobiti tačan rezultat, što je još jedna od novina C++11 standarda).

76.11.2.1 Format obrađivača

Opšti oblik C++ obrađivača izuzetaka je:

```
try {
    -- dio koda koji može podići izuzetak
  }
catch (formal parameter) {
    -- obrađivač
  }
...
catch (formal parameter) { -- obrađivač
  }
```

Pri nailasku na ovu konstrukciju, prvo se izvršava skupina naredbi unutar **try**-bloka koja može dići izuzetak. Ukoliko prilikom izvršavanja ovih naredbi ne dođe ni do kakvog bacanja izuzetaka, tj. ukoliko se sve naredbe okončaju na regularan način, skupina koje hvataju izuzetke neće se uopće izvršiti, odnosno program će nastaviti izvršavanje iza zatvorene vitičaste zagrade koja označava kraj ovog bloka (tj. iza cijele **try** – **catch** konstrukcije) i taj **try**-blok više neće biti aktivan. Međutim, ukoliko prilikom izvršavanja naredbi u bloku "try" dođe do bacanja izuzetka (npr. pozove se neka funkcija koja baci izuzetak), izvršavanje svih daljih akcija koje eventualno slijede u bloku "**try**" se istog trenutka prekida, a izvršavanje programa se nastavlja od prve naredbe u skupini za hvatanje izuzetaka (tačnije, od prve naredbe unutar **catch**-bloka koji odgovara **try**-bloku koji je u tom trenutku bio aktivan). Iza naredbe "**catch**" u zagradama se mora nalaziti deklaracija jednog formalnog parametra, koji će prihvatiti vrijednost bačenu naredbom "**throw**".

Na taj način, naredbom "throw" može se "baciti" vrijednost koja će biti "uhvaćena" u naredbi "**catch**". Obrađivači izuzetaka se navode na mjestu gdje je i kod koji može podići izuzetak. Što se tiče opsega, često se dešava da jedna funkcija poziva drugu, druga treću, treća četvrtu itd. Pretpostavimo na primjer, da neka od funkcija u lancu pozvanih funkcija (npr. treća) baci izuzetak. Prirodno je postaviti pitanje gdje će takav izuzetak biti uhvaćen. Odgovor se krije u činjenici da se tok programa nastavlja unutar onog **catch**-bloka koji odgovara onom **try**-bloku koji je bio aktivan u trenutku bacanja izuzetka. Međutim, ukoliko se pri izvršavanju programa naiđe na novi **try**-blok dok je neki drugi **try**-blok još uvijek aktivan, taj novi **try**-blok će postati aktivan umjesto starog. Nakon završetka tog novog **try**-bloka, ponovo će biti aktivan onaj prethodni **try**-blok koji je bio aktivan prije nailaska na novi **try**-blok.

76.11.2.2 Prosljeđivanje obrađivaču

Smatra se lijepom praksom da se kao izuzeci ne bacaju jednostavni objekti tipa "**int**" i slično, nego da se za tu svrhu koriste posebni tipovi za modeliranje izuzetaka, definirani u biblioteci "stdexcept". Ima desetak takvih tipova kao što su "logic_error", "domain_error", "overflow_error" itd. a svi se kreiraju konverzijom iz niza znakova u odgovarajući tip. Ideja je da se pogodnim tipom izuzetka pobliže modelira o kakvoj vrsti problema se radi. Na primjer, u skladu sa tom konvencijom, funkcija "Faktorijel" mogla bi se napisati ovako:

```
long long int Faktorijel(int n) {
    if(n < 0) throw std::domain_error("Pogresan argument!");
    long long int p(1);
    for(int i = 1; i <= n; i++) p *= i;
    return p;
  }
```

Vidimo da smo specificirali tip izuzetka a i uz izuzetak poslali informaciju zašto je došlo do izuzetka („Pogresan parametar“). Pri tome, tekst koji smo iskoristili za kreiranje izuzetka možemo dobiti pozivom funkcije “**what**” (bez parametara) nad objektom koji modelira izuzetak. Recimo, izuzetke bačene iz prethodne funkcije mogli bismo hvatati i obrađivati ovako:

```
try {
    int n;
    std::cin >> n;
    std::cout << n << "! = " << Faktorijel(n) << std::endl;
  }
catch(std::domain_error izuzetak) {
    std::cout << izuzetak.what() << std::endl;
  }
```

76.11.2.3 Nastavak izvršavanja

Ukoliko se prilikom izvršavanja neke funkcije naiđe na naredbu “**throw**”, njeno izvršavanje se prekida (osim u slučaju da se naredba “**throw**” nalazila unutar **try**-bloka, što je specijalni slučaj koji ćemo razmotriti kasnije) i tok programa se vraća na mjesto poziva funkcije. Pri tome se dešavaju iste stvari kao i pri regularnom završetku funkcije (npr. sve lokalne promjenljive funkcije bivaju uništene). Dalji tok programa sada bitno ovisi da li je funkcija koja je bacila izuzetak pozvana iz unutrašnjosti nekog **try**-bloka ili nije. Ukoliko je funkcija pozvana iz **try**-bloka, izvođenje naredbe unutar koje se nalazi poziv funkcije i svih naredbi koje eventualno slijede iza nje u **try**-bloku se prekida, i tok programa se preusmjerava na početak odgovarajućeg **catch**-bloka, kao što je već opisano.

Međutim, ukoliko funkcija koja je bacila izuzetak nije pozvana iz nekog **try** -bloka, izvršavanje funkcije iz koje je problematična funkcija pozvana biće također prekinuto, kao da je upravo ona bacila izuzetak, i kontrola toka programa se privremeno prenosi na mjesto poziva te funkcije. Postupak se dalje ponavlja sve dok izuzetak ne bude uhvaćen u nekoj od funkcija u lancu pozvanih funkcija, nakon čega se tok programa nastavlja u pripadnom **catch**-bloku (koji odgovara mjestu gdje je izuzetak uhvaćen). Ukoliko se u toku ovog “razmotavanja” poziva dođe i do same “**main**” funkcije, a izuzetak ne bude uhvaćen ni u njoj, program će biti prekinut. Iz gore izloženog vidimo da se svaki izuzetak koji ne bude uhvaćen unutar neke funkcije prosljeđuje dalje na mjesto poziva funkcije, sve dok ne bude eventualno uhvaćen.

Poslije uspješne obrade izuzetka, obrada nastavlja regularnim tokom, tj. ne vraća se već izvršava sljedeću instrukciju koja je napisana u programu.

```
try {
    return Faktorijel(n) / (Faktorijel(k) * Faktorijel(n - k));
  }
catch(...) {
    return 0;
  }
```

A u ovom primjeru je unutar **catch**-a **return** 0 tako da će se izaći iz ove funkcije sa rezultatom 0, da nije bilo **return**-a naredbe iza vitičastih zagrada bi se nastavile izvršavati.

Klauzula finally može se pojaviti na kraju **try** konstrukcije. Namjena je da se navede kod koji će se izvršiti bez obzira šta se dešava u **try** konstrukciji. Njen oblik je

```
finally {
    ...
  }
```

76.11.2.4 Korisnički definisani izuzeci

Korisnički definisani izuzeci moraju dolaziti (nasljeđivati) iz std::exception klase i preklopiti (override) metodu **what**(). Neka je npr. definisan korisnički izuzetak.

```
class Exception : public exception
  {
    public:
    Exception(const char* errMessage):errMessage_(errMessage){}
    // preklopljen what() metod iz klase exception
    const char* what() const throw() { return errMessage_; }

    private:
    const char* errMessage_;
  };
```

76.11.2.5 Standardni izuzeci

Nepraktična osobina C++ je što izuzeci nisu imenovani. Nema predefinisanih hardverskih i sistemskih izuzetaka. Vezivanje izuzetaka za obrađivače kroz tip parametara ne popravlja čitljivost.

Nekoliko standardnih izuzetaka postoje definirani u biblioteci "stdexcept". Ima desetak takvih tipova kao što su "logic_error", "domain_error", "overflow_error" itd. a svi se kreiraju konverzijom iz niza znakova u odgovarajući tip.

76.11.3 Programiranje vođeno događajima

Standard C++ uopšte ne radi sa događajima i nema ugrađenu podršku za upravljanje događajima, međutim ukoliko trebamo događaje radimo u okviru frejmvorka koji ih obezbjeđuju (SDL, Windows, Qt, GNOME, itd.)

76.12 Podrška drugim paradigmama

Funkcionalni i deklarativni programski jezici imaju drugačiji koncept programiranja od imperativnog. Ipak, sve češće neki njihovi elementi ulaze kao dopuna imperativnim jezicima, kao što su anonimne funkcije, operatori nad funkcijama.

76.12.1 Podrška funkcionalnoj paradigmi

U C++11 uvedene su tzv. anonimne funkcije, koje su poznate i pod nazivom λ funkcije odnosno lambda funkcije (naziv potiče od istoimenog koncepta koji je u matematiku uveo logičar A. Church u prvoj polovini 2 . vijeka). Lambda funkcije su funkcije koje imaju popis parametara i tijelo, ali nemaju imena. U jeziku C++ (od verzije C++11) takve funkcije se kreiraju tako što se umjesto imena funkcije stavi par uglastih zagrada "[]" (one su zasada prazne, mada ćemo kasnije vidjeti da se i unutar njih može nalaziti nešto). Također, kod lambda funkcija ne navodi se tip povratne vrijednosti, nego se on automatski određuje na osnovu tipa izraza koji se nalazi iza "**return**" naredbe unutar njenog tijela. Ukoliko ipak moramo navesti tip povratne vrijednosti (a moramo ukoliko se unutar tijela nalazi više "**return**" naredbi iza kojih se nalaze

izrazi različitih tipova, ili ukoliko iz bilo kojeg razloga želimo da tip povratne vrijednosti bude drugačiji od tipa onoga što je stavljeno iza "**return**"), tada tip povratne vrijednosti stavljamo iza spiska parametara, koristeći sintaksu sa znakom "->" koju smo već objašnjavali. Slijedi primjer dvije ispravne lambda funkcije (kako su tijela lambda funkcija najčešće vrlo kratka, lambda funkcije se često kompletne pišu u jednom redu):

```
[](double x) { return x * x * x - x - 1; }
[](int x, double y) -> double { if(x > y) return x; else return y; }
```

Strogo rečeno, standard C++ propisuje da se tip povratne vrijednosti mora navesti čim se tijelo lambda funkcije sastoji i od čega drugog osim jedne jedine "return" naredbe. Ovo je posve nepotrebno ograničenje i mnogi kompajleri ga ne poštuju. Komitet za standardizaciju jezika C++ je također postao svjestan besmislenosti ovog ograničenja, tako da je ono uklonjeno u novoj maloj reviziji standarda poznatoj kao C++14. Lambda funkcije nemaju imena, pa ih nije moguće pozvati po imenu. Njihova uloga je zapravo da se koriste unutar izraza. Naime, gdje god sintaksa jezika C++ očekuje ime neke funkcije, na tom mjestu je moguće staviti lambda funkciju (kompletnu, skupa sa njenim tijelom), i ona će biti pozvana. To nam omogućava da na mjestu gdje se očekuje ime neke funkcije, tu funkciju definiramo na licu mjesta, umjesto da je definiramo i imenujemo negdje drugdje, pa da koristimo njeno ime. Recimo, sasvim je legalna (u C++) sljedeća neobična konstrukcija, i ona će ispisati broj 15 na ekran:

```
std::cout << 3 + 2 * [](int x, int y) { return 2 * x - y; } (5, 2) - 4;
```

Razmotrimo sada što se tačno dešava prilikom prenosa funkcija kao parametara drugim funkcijama. Ono što je činjenica je da se tom prilikom zapravo ne prenosi čitava funkcija nego samo njena adresa (tačnije adresa mašinskog potprograma koji realizira tu funkciju). Da bismo detaljnije objasnili taj mehanizam, moramo se upoznati sa pokazivačima na funkcije, ali to zahtijeva da se prvo malo podsjetimo šta su prototipovi funkcija (poznati još kao deklaracije funkcija ili funkcijske deklaracije). Znamo da u tipičnim situacijama svaka funkcija koju pozivamo treba biti definirana prije mjesta njenog poziva. Prototipovi funkcija nam omogućavaju da funkcije definiramo iza mjesta poziva funkcije, pa čak i u nekoj drugoj programskoj datoteci. Prototipovi funkcija informiraju kompajler koliko parametara prihvata funkcija, kakvi su tipovi parametara, i koji je tip povratne vrijednosti funkcije. Oni se sastoje samo od zaglavlja funkcije, bez njenog tijela (umjesto tijela stavlja se tačka-zarez). Na primjer, sljedeća deklaracija deklariše funkciju dva cjelobrojna argumenta i vraća cjelobrojnu vrijednost kao rezultat.

```
int fun(int x, int y);
// Prototip funkcije fun predstavlja prototip funkcije "" fun
//koja prima dva cjelobrojna
```

S obzirom da su u prototipovima imena argumenata posve nebitna, mogu se izostaviti, tako da smo isti prototip mogli napisati i ovako:

```
int fun(int, int); // I ovo je ispravan prototip
```

C++ ne podržava izvorno lijenu evaluaciju, ali ju je moguće implementirati, što se i radi u određenim situacijama (npr. za rad sa matricama).

76.12.2 Podrška za logičko i deklarativno programiranje

Jezici za logičko programiranje su vrsta deklarativnih programskih jezika. Programi u logičkim jezicima se izražavaju u formi simboličke logike. Izvršni sistemi ovih jezika koriste proces logičkog zaključivanja da dođu do rezultata . Kako C++ ne pripada skupini logičkih programskih jezika, nego skupini imperativnih (objektno orijentiranih), to znači da nije moguće pisati programe isključivo od deklaracija.

76.13 Bibliografija

[1] Michel Goossens, Frank Mittelbach, and Alexander Samarin. *The LaTeX Companion*. Addison-Wesley, Reading, Massachusetts, 1993.

[2] Albert Einstein. *Zur Elektrodynamik bewegter Körper*. (German) [*On the electrodynamics of moving bodies*]. Annalen der Physik, 322(10):891–921, 1905.

[3] Knuth: Computers and Typesetting,
`http://www-cs-faculty.stanford.edu/~uno/abcde.html`